Computational Complexity
P vs NP (Lec14.pdf)

Ioannis Akrotirianakis

Todays topics & Logistics

- Introduction to Computational Complexity
- Basic definitions
- Examples
- Reductions

We will use these slides for the remaining lectures

Information about the final exam will be announced this week!

Computational Complexity Theory

- **What is it?** A branch of Mathematics and Computer Science that provides a formal framework for studying how efficiently we can solve computational problems on a computer.
- **Crucial for Optimization & Computational Sciences!** Because we are interested in developing algorithms to solve problems as fast as possible
- So far, we have learned know that Convex Optimization problems can be solved efficiently (fast) so they are "easy".
- But what about NonConvex Optimization problems? Are they "easy" or "hard" to solve?
- Are there any Convex Optimization problems that are not "easy" or "hard" to solve?
- **What does exactly mean: "a problem is "hard" to solve??"**

Computational Complexity Theory

- There are 3 types of problems (definitions below for the Max Stable Set):

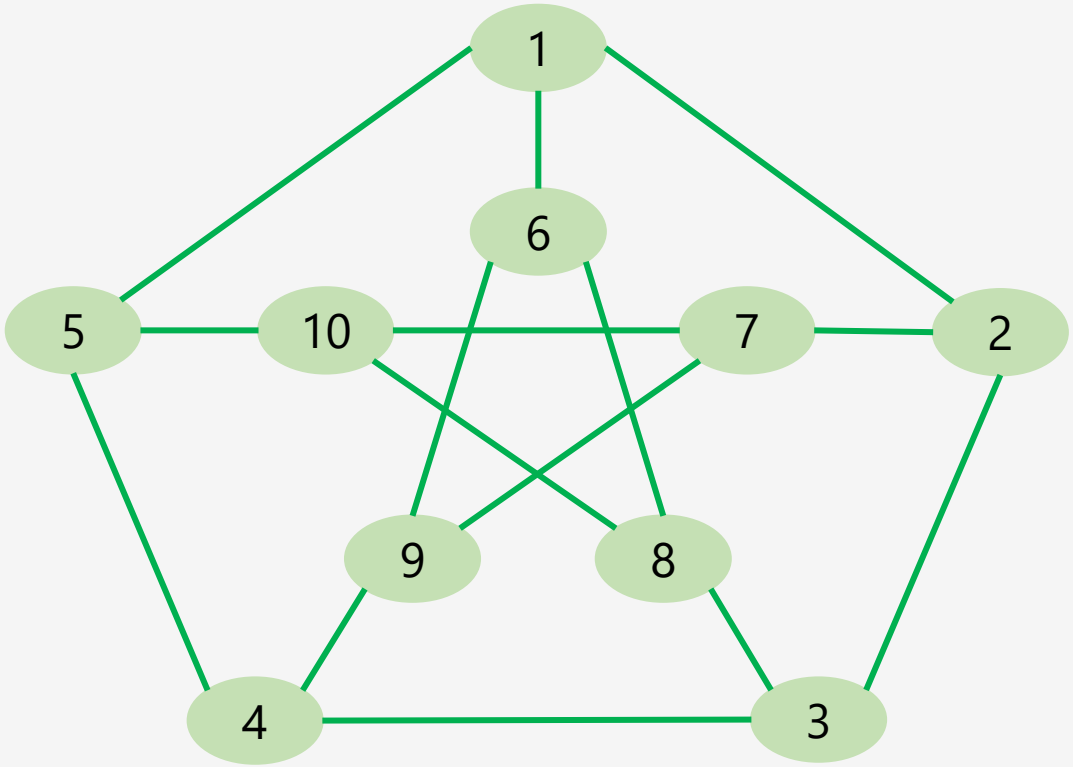
- **Optimization problem**: given a graph G, **compute** its largest stable set → **the output to this problem is a number (the value of the objective function at an optimum**

$\Updownarrow$

- **Decision problem**: given a graph G and an integer b, **decide** if there exists a stable set of size $\geq b$? → **the output to this problem is YES or NOT**

$\Updownarrow$

- **Search problem**: given a graph G and an integer b, find a stable set of size $\geq b$ or declare that none exists → **the output to this problem is a set or a 0-1 number**

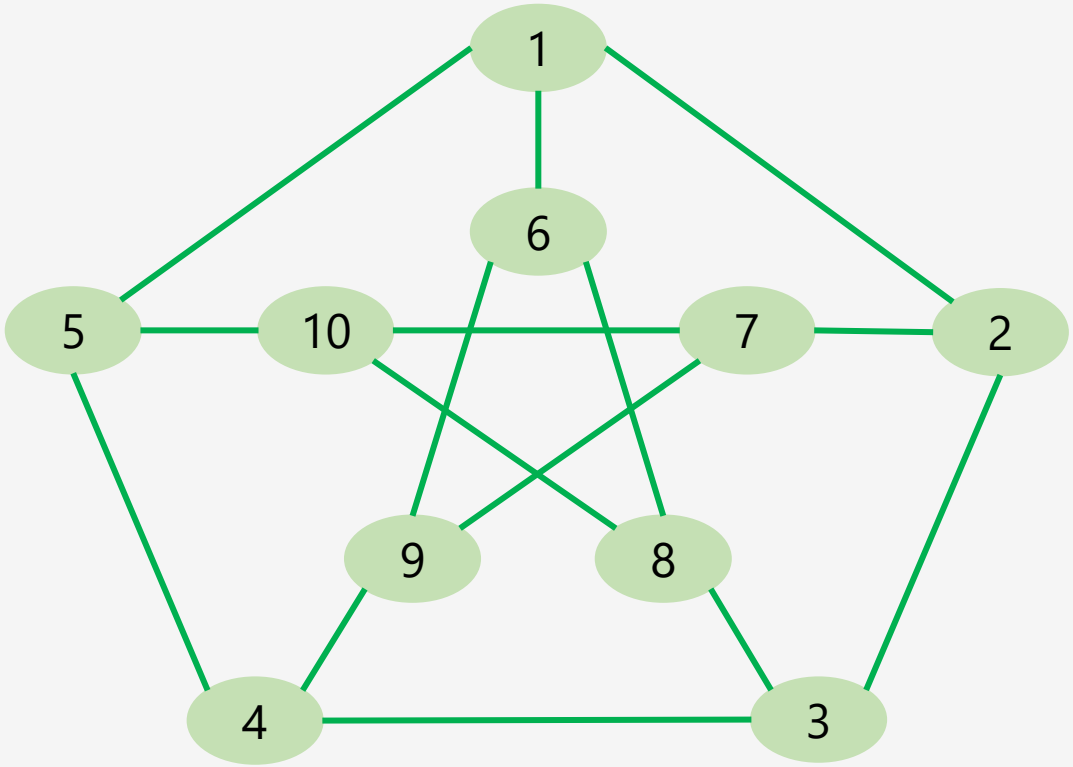


Decision problem –vs- Instance

- A **decision problem** is a general description of the problem we want to solve
  - It has a finite input that is given to us (data). The input is characterized by a set of vertices V and a set of edges E
- **Instance** of a decision problem is a specific case of the problem. The vertex and edge sets have specific values.
  - For example: $V = \{1,2,\dots,10\}$

    $E = \{(1,2),(2,3),\dots,(10,8)\}$

Petersen graph **instance** of the STABLESET decision problem

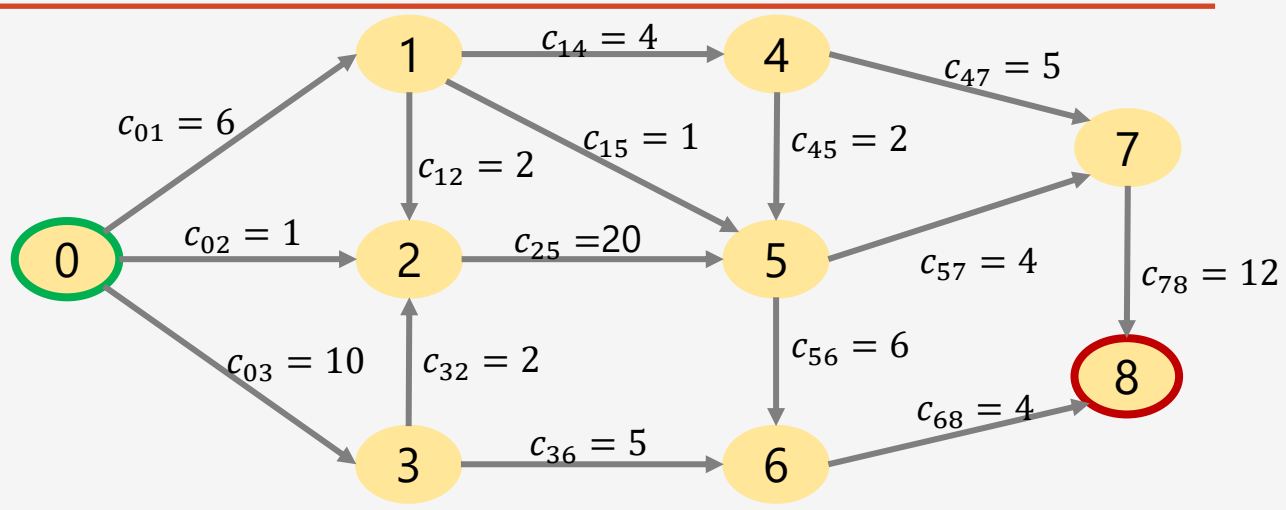


Examples of Decision problems

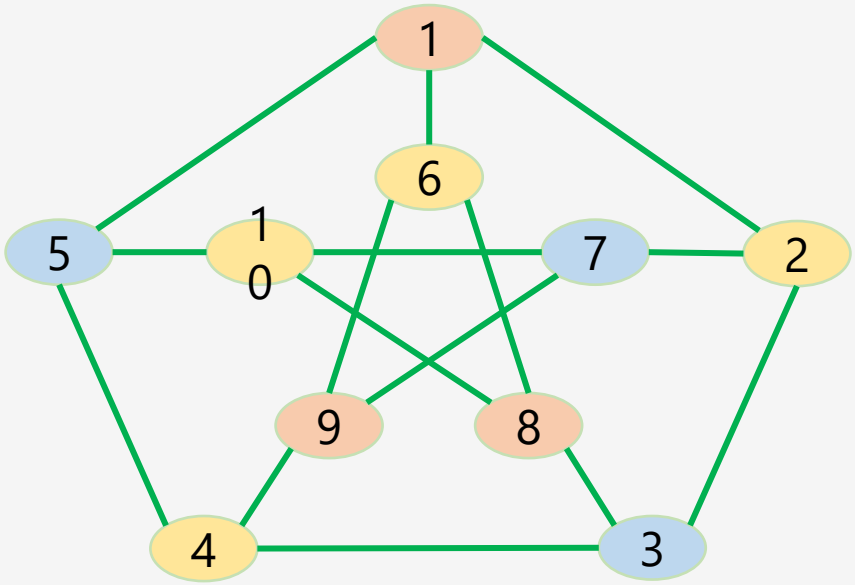
- **LINEQ:**
  - Input: $A \in \mathbb{Q}^{m \times n}$, $b \in \mathbb{Q}^{n}$
  - Question: is there a solution to the linear system: $Ax = b$
- **ZOLINEQ:**
  - Input: $A \in \mathbb{Q}^{m \times n}$, $b \in \mathbb{Q}^{n}$
  - Question: is there a 0-1 solution to the linear system: $Ax = b$
- **LP:**
  - Input: $c \in \mathbb{Q}^{m}$, $A \in \mathbb{Q}^{m \times n}$, $b \in \mathbb{Q}^{n}$, $k \in \mathbb{Q}$
  - Question: is there an optimal value of the LP $\leq \mathrm{k}$
- **IP:**
  - Input: $c \in \mathbb{Q}^{m}$, $A \in \mathbb{Q}^{m \times n}$, $b \in \mathbb{Q}^{n}$, $k \in \mathbb{Q}$
  - Question: is there an integer feasible solution to the LP with objective value $\leq \mathrm{k}$

Examples of Decision problems

- **MAXFLOW:**
  - Input: a directed graph $G = (V, E)$, $c_{ij} \in \mathbb{Q}$ cost on edge (i, j), start node S, end node T, $k \in \mathbb{Q}$
  - Question: is there a flow of value $\geq k$ from S to T that respects the edge costs & flow conservation constraints

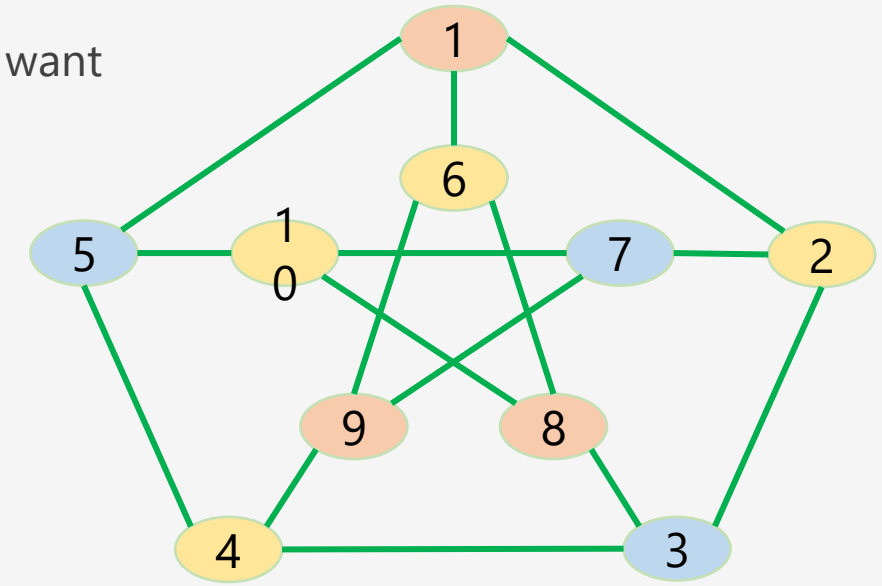


- **COLORING:**
  - Input: undirected graph $G = (V, E)$ and $k \in \mathbb{Q}$
  - Question: is the graph k-colorable?



Size of an Instance

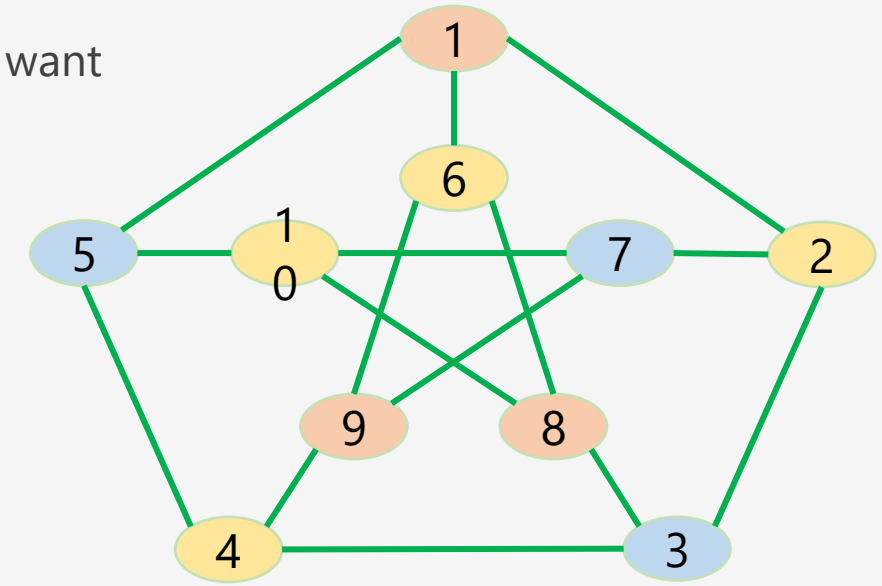
Peterson graph instance



- The running time of an algorithm depends on the size of the problem we want to solve → size of the "Instance"
- An algorithm may take longer for a larger instances
- COLORING:
  - Input: undirected graph $G = (V, E)$ and $k \in \mathbb{Q}$
  - **Possible candidates for input size ??**

Size of an Instance

Peterson graph instance



- The running time of an algorithm depends on the size of the problem we want to solve → size of the "Instance"
- An algorithm may take longer for a larger instances
- COLORING:
  - Input: undirected graph $G = (V, E)$ and $k \in \mathbb{Q}$
  - **Possible candidates for input ??**
    - Number of bits needed to store the vertices
    - Number of bits needed to store the nodes & edges
    - Number of bits needed to store the **adjacency matrix** A of the graph

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 0 \end{bmatrix}$$

Size of an instance

- **Instance size (or input size)** can be thought as the number of bits required to store/represent the input

- LP:

  - Input: $c \in \mathbb{Q}^{m}$, $A \in \mathbb{Q}^{m \times n}$, $b \in \mathbb{Q}^{n}$, $k \in \mathbb{Q}$

  - Question: is there an optimal value of the LP $\leq k$

What is the input size ??

Hint1: how many bits integer numbers do we have in total in c, A, b, k ??

Hint2: How many bits we need to represent an integer ??

Size of an instance

- **Instance size (or input size)** can be thought as the number of bits required to store/represent the input

- LP:

  - Input: $c \in \mathbb{Q}^{\mathrm{m}}$, $A \in \mathbb{Q}^{\mathrm{m}\times\mathrm{n}}$, $b \in \mathbb{Q}^{\mathrm{n}}$, $k \in \mathbb{Q}$

  - Question: is there an optimal value of the LP $\leq \mathrm{k}$

- The input size (instance size) is bounded by the number:

$$\underset{A}{\boldsymbol{2(mn}}\underset{c}{\boldsymbol{+m}}\underset{b}{\boldsymbol{+n}}\underset{k}{\boldsymbol{+1)}}\ \boldsymbol{ceil(\log_2(L+1))}$$

  - L is the largest integer appearing in the numerator or denominator of any entry of c, A, b, k

Size of an instance

- **Instance size (or input size)** can be thought as the number of bits required to store/represent the input

- LP:
  - Input: $c \in \mathbb{Q}^{\mathrm{m}}$, $A \in \mathbb{Q}^{\mathrm{m}\times\mathrm{n}}$, $b \in \mathbb{Q}^{\mathrm{n}}$, $k \in \mathbb{Q}$
  - Question: is there an optimal value of the LP $\leq$ k

Example: How many bits we need to represent 10 in the binary system ??

4 bits $\leftrightarrow$ ceil(log(10 + 1))=ceil(3.459)=4

$$10 \leftrightarrow 1010 \leftrightarrow 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0$$

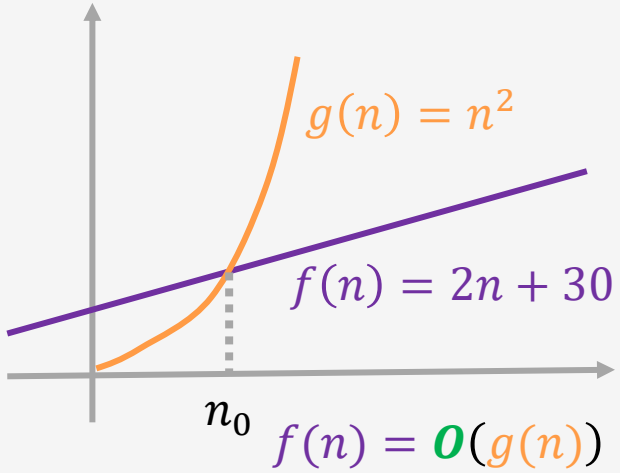
- The input size (instance size) is bounded by the number:

$$\underbrace{\mathbf{2}(\boldsymbol{mn}}_{A} + \underbrace{\boldsymbol{m}}_{c} + \underbrace{\boldsymbol{n}}_{b} + \underbrace{\mathbf{1})}_{k}\ \boldsymbol{ceil}(\mathbf{log_2}(\boldsymbol{L} + \mathbf{1}))$$

  - L is the largest integer appearing in the numerator or denominator of any entry of c, A, b, k

Running times of algorithms

- **Definitions**: Let $f, g: R_+ \to R_+$.
  - $f(n) = \boldsymbol{O}(g(n))$, if $\exists n_0, c > 0$, s.t. $f(n) \leq cg(n)$, $\forall n \geq n_0$ — g dominates f
  - $f(n) = \boldsymbol{\Omega}(g(n))$, if $\exists n_0, c > 0$, s.t. $f(n) \geq cg(n)$, $\forall n \geq n_0$ — f dominates g
  - $f(n) = \boldsymbol{\Theta}(g(n))$, if $\exists n_0, c > 0$, s.t. $c_1 g(n) \leq f(n) \leq c_2 g(n)$, $\forall n \geq n_0$ — same growth rate
- **Examples**:
  1. $5n^3 + 2n^2 + 40 = \Theta(n^3)$,
  2. $n \log n = O(n^2)$,
  3. $n \log n = \Omega(n)$,
  4. $2^{cn} = \Omega(n^k)$



Polynomial & Exponential time algorithms

- **Polynomial time algorithm** : running time as a function of its input size, n, is $O(p(n))$, for some polynomial $p: R \to R$ (also $O(n^c)$, where $c = \deg(p)$)
  - Usual notation: $O(n^c)$, where $c = \deg(p)$
- **Exponential time algorithm** : running time as a function of its input size, n, is $\Omega(2^{cn})$, for some $c > 0$
- When we talk about the **running time of an algorithm**, we mean the **worst-case running time** over all inputs of size n
- Poly-time: Exp-time:

Exponential time algorithms can be very slow!

- 2^n can grow very fast!

2^0	2^1	2^2	2^3	2^4	2^5	2^6	2^7
2^8	2^9	2^{10}	2^{11}	2^{12}	2^{13}	2^{14}	2^{15}
2^{16}	2^{17}	2^{18}	2^{19}	2^{20}	2^{21}	2^{22}	2^{23}
2^{24}	2^{25}	2^{26}	2^{27}	2^{28}	2^{29}	2^{30}	2^{31}
2^{32}	2^{33}	2^{34}	2^{35}	2^{36}	2^{37}	2^{38}	2^{39}
2^{40}	2^{41}	2^{42}	2^{43}	2^{44}	2^{45}	2^{46}	2^{47}
2^{48}	2^{49}	2^{50}	2^{51}	2^{52}	2^{53}	2^{54}	2^{55}
2^{56}	2^{57}	2^{58}	2^{59}	2^{60}	2^{61}	2^{62}	2^{63}

$2^{64} \sim 18 \times 10^{18}$ grains are too much to ask!!

Sissa was an ancient Indian mathematician who discovered Chaturanga which is similar to Chess.

Comparison of running times

Time complexity function	Size n					
	10	20	30	40	50	60
n	.00001 second	.00002 second	.00003 second	.00004 second	.00005 second	.00006 second
n^2	.0001 second	.0004 second	.0009 second	.0016 second	.0025 second	.0036 second
n^3	.001 second	.008 second	.027 second	.064 second	.125 second	.216 second
n^5	.1 second	3.2 seconds	24.3 seconds	1.7 minutes	5.2 minutes	13.0 minutes
2^n	.001 second	1.0 second	17.9 minutes	12.7 days	35.7 years	366 centuries
3^n	.059 second	58 minutes	6.5 years	3855 centuries	2×10^8 centuries	1.3×10^{13} centuries

(image credit: [GJ79])

Polynomial times algorithms need seconds or minutes to complete

But note the **dramatic increase** in running time (up to **centuries**…) of Exponential time algorithms!

Can Moore's law help us?

Size of Largest Problem Instance Solvable in 1 Hour

Time complexity function	With present computer	With computer 100 times faster	With computer 1000 times faster
n	N_1	$100\ N_1$	$1000\ N_1$
n^2	N_2	$10\ N_2$	$31.6\ N_2$
n^3	N_3	$4.64\ N_3$	$10\ N_3$
n^5	N_4	$2.5\ N_4$	$3.98\ N_4$
2^n	N_5	$N_5+6.64$	$N_5+9.97$
3^n	N_6	$N_6+4.19$	$N_6+6.29$

Effect of **improved technology** on several Polynomial and Exponential time algorithms

(image credit: [GJ79])

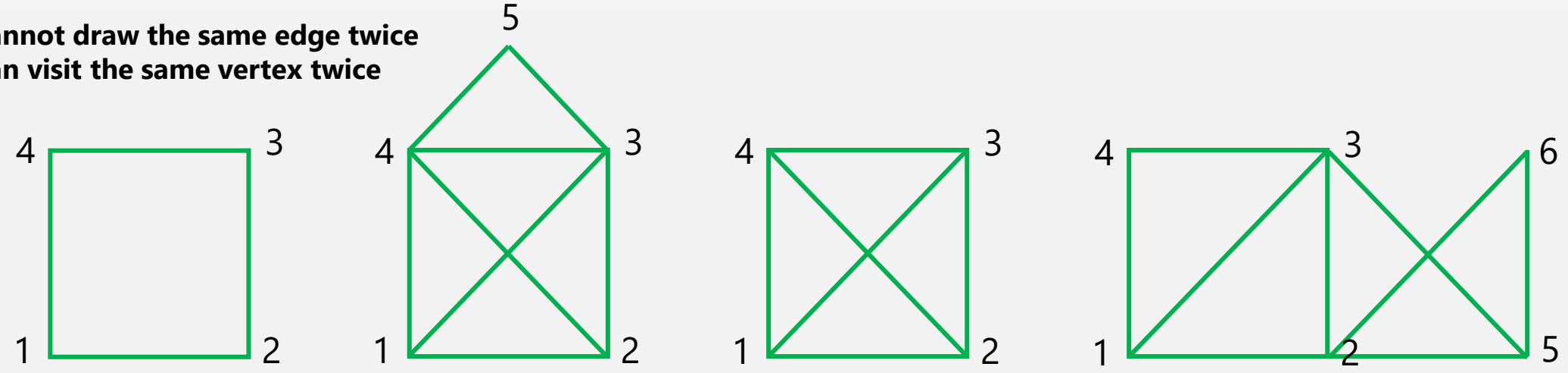
Complexity class P

- The class of Decision problems that admit a poly-time algorithm

$$\boldsymbol{P} = \left\{ \begin{array}{c} \text{ADDITION, MULTIPLICATION, LINEQ, LP, MAXFLOW, MINCUT,} \\ \text{MATRIXPOS, SHORTEST, PATH, SDP}\epsilon\text{, PRIMES, ZEROSUMNASH, PENONPAPER,...} \end{array} \right\}$$

- **Example**: PENONPAPER
  - **<u>Input</u>**: A connected undirected graph; **<u>Question</u>**: Can you draw the graph without lifting your

You cannot draw the same edge twice
You can visit the same vertex twice



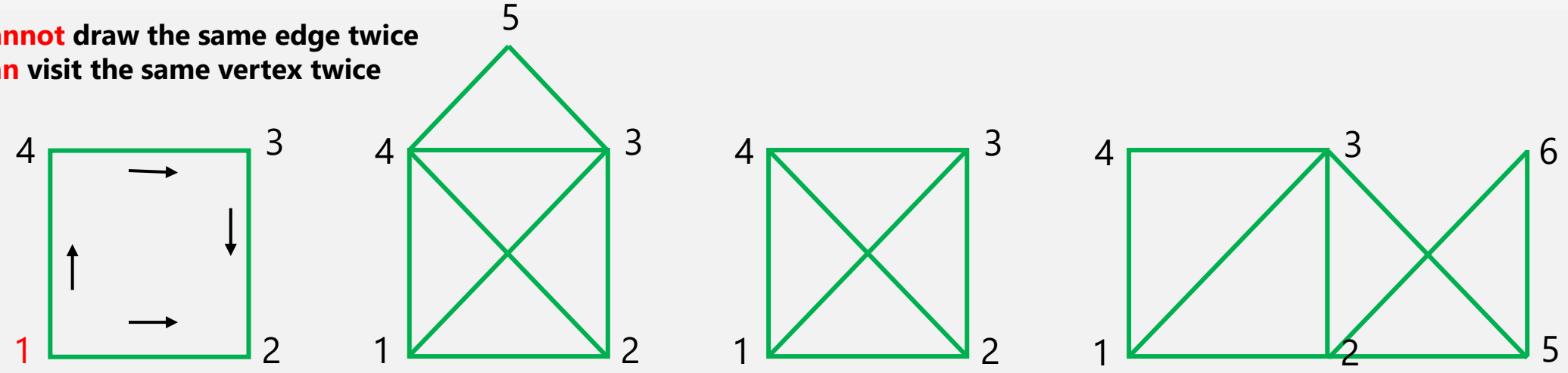
Complexity class P

- The class of Decision problems that admit a poly-time algorithm

$$\boldsymbol{P} = \left\{ \begin{array}{c} \text{ADDITION, MULTIPLICATION, LINEQ, LP, MAXFLOW, MINCUT,} \\ \text{MATRIXPOS, SHORTEST, PATH, SDP}\epsilon\text{, PRIMES, ZEROSUMNASH, PENONPAPER,...} \end{array} \right\}$$

- **Example**: PENONPAPER
  - **<u>Input</u>**: A connected undirected graph; **<u>Question</u>**: Can you draw the graph without lifting your

You cannot draw the same edge twice
You can visit the same vertex twice



Complexity class P

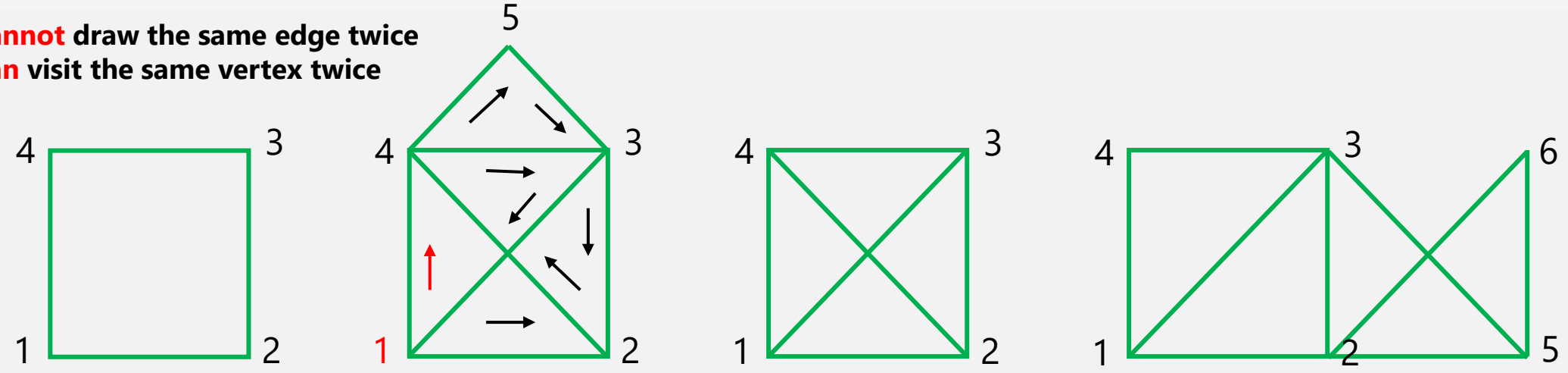
- The class of Decision problems that admit a poly-time algorithm

$$\boldsymbol{P} = \left\{ \begin{array}{c} \text{ADDITION, MULTIPLICATION, LINEQ, LP, MAXFLOW, MINCUT,} \\ \text{MATRIXPOS, SHORTEST, PATH, SDP}\epsilon\text{, PRIMES, ZEROSUMNASH, PENONPAPER,...} \end{array} \right\}$$

- **Example**: PENONPAPER

  - **<u>Input</u>**: A connected undirected graph; **<u>Question</u>**: Can you draw the graph without lifting your

You cannot draw the same edge twice
You can visit the same vertex twice



Complexity class P

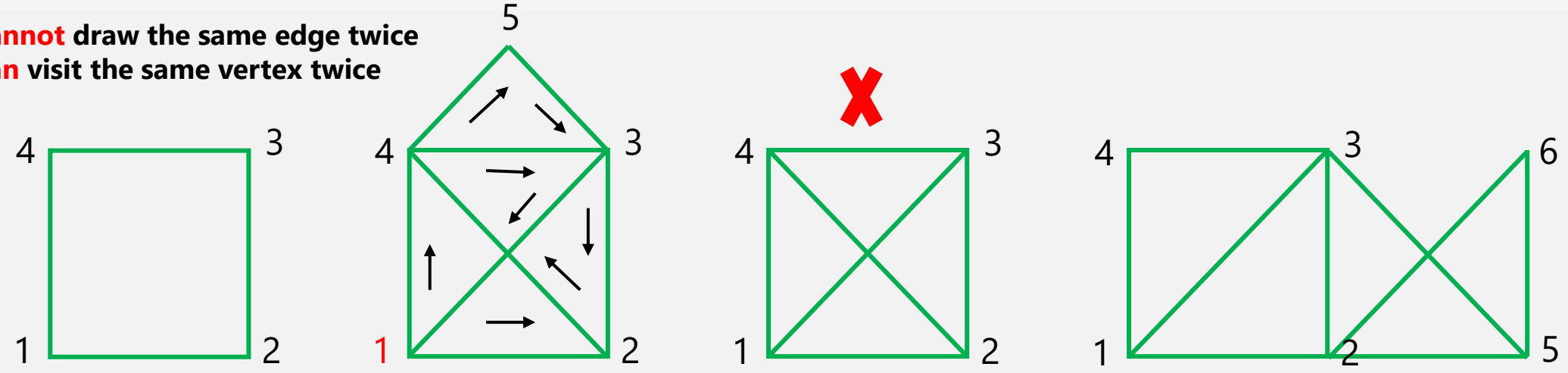
- The class of Decision problems that admit a poly-time algorithm

$$\boldsymbol{P} = \left\{ \begin{array}{c} \text{ADDITION, MULTIPLICATION, LINEQ, LP, MAXFLOW, MINCUT,} \\ \text{MATRIXPOS, SHORTEST, PATH, SDP}\epsilon\text{, PRIMES, ZEROSUMNASH, PENONPAPER,...} \end{array} \right\}$$

- **Example**: PENONPAPER

  - Input: A connected undirected graph; Question: Can you draw the graph without lifting your pen?

You cannot draw the same edge twice
You can visit the same vertex twice



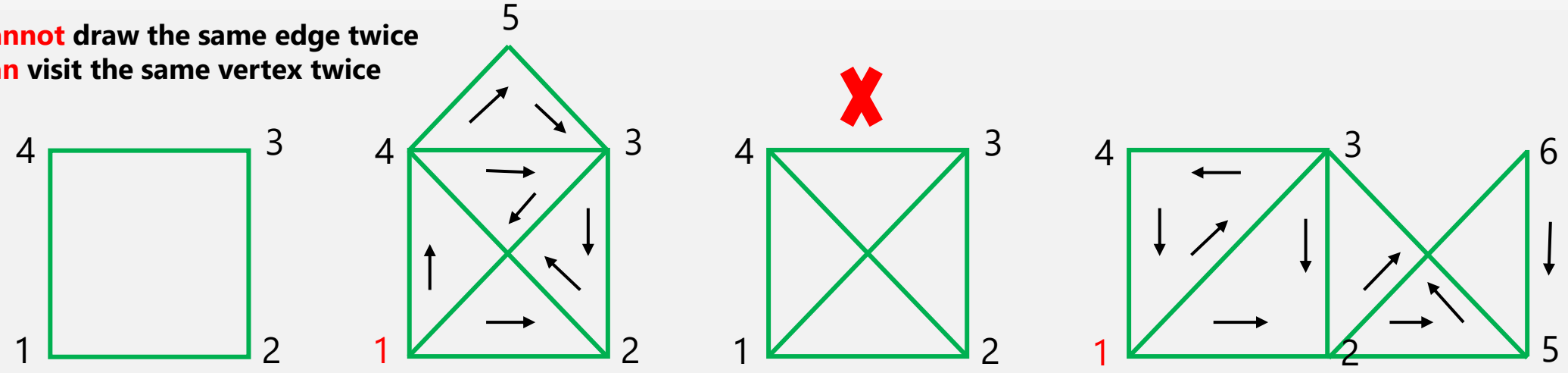
Complexity class P

- The class of Decision problems that admit a poly-time algorithm

$$\boldsymbol{P} = \left\{ \begin{array}{c} \text{ADDITION, MULTIPLICATION, LINEQ, LP, MAXFLOW, MINCUT,} \\ \text{MATRIXPOS, SHORTEST, PATH, SDP}\epsilon\text{, PRIMES, ZEROSUMNASH, PENONPAPER,...} \end{array} \right\}$$

- **Example**: PENONPAPER
  - Input: A connected undirected graph; Question: Can you draw the graph without lifting your pen?

You cannot draw the same edge twice
You can visit the same vertex twice

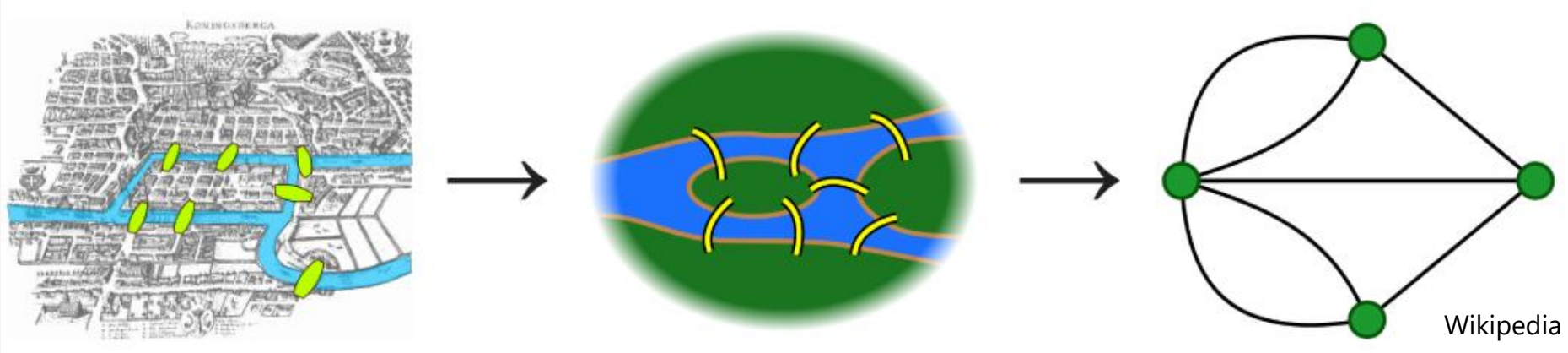


Complexity class P

- The class of Decision problems that admit a poly-time algorithm

$$\boldsymbol{P} = \left\{ \begin{array}{c} \text{ADDITION, MULTIPLICATION, LINEQ, LP, MAXFLOW, MINCUT,} \\ \text{MATRIXPOS, SHORTEST, PATH, SDP}\epsilon\text{, PRIMES, ZEROSUMNASH, PENONPAPER,...} \end{array} \right\}$$

- **Example**: **PENONPAPER**
  - **Input**: A connected undirected graph; **Question**: Can you draw the graph without lifting your pen?
  - **Euler** was the first to answer PENONPAPER as **YES**, **if and only if**, **every node,** with the possible exception of two nodes, **has even degree**.
  - This condition can be checked in Poly-time (just count the degree of nodes). → PENONPAPER $\in P$



Historical note:

Euler was wondering if it is possible to walk across the city of **Konigsberg** (present day Kaliningrad) and **cross each of the 7 bridges once.**

He proved it is **impossible**

Taking walks is not only healthy, but it may help you discover a new branch of Mathematics → **Graph Theory**

Some useful known Poly-time Decision problems

- **LINEQ** (solve a system of linear equalities)

  - Gaussian elimination or Conjugate Gradient methods are $O(n^3)$

- **LP** (solve a system of linear inequalities)

The New York Times

  - Ellipsoid [Khachiyan, 1979] & Interior Point [Karmarkar, 1984] algorithms

https://www.nytimes.com/1979/11/27/archives/soviet-mathematician-is-obscure-no-more-mathematician-is-obscure-no.html https://www.nytimes.com/1984/11/19/us/breakthrough-in-problem-solving.html

- **PRIMES** (decide if a given integer is prime)

  - Proved recently [Agrawal, Kayal, Saxena, 2002] The New York Times

Graduate students

Reductions

- **Reductions** play a fundamental role in Complexity Theory
- Many **new** Decision problems are shown to be Poly-time ($\in P$) by **reducing them** to Decision problems **we already know** they are Poly-time ($\in P$)
- **A reduces to B means: "if we could do B, then we could do A"**
  - Notation: A → B
  - Well-known joke about Mathematicians and Engineers

Well-known joke about Mathematicians and Engineers

- Mathematicians think of Reductions too often, even when they asked to boil water…

The Engineer

Day 1

Day 2

The Mathematician

Reductions

- **More formally**: A **Reduction** from a Decision problem A to a Decision problem B is →
  - a general recipe (aka, algorithm) for taking an instance of A and explicitly producing an instance of B, such that →
    - the answer to instance A is YES, **iff** the answer to the produced instance of B is YES.
- This definition enables us to **answer A by answering B**

A → B

Example of a Reduction: MAXFLOW → LP

- **Example**: Show that MAXFLOW is Poly-time that is, **MAXFLOW** $\in \boldsymbol{P}$
  - We need to find some Decision problem B that we already know is Poly-time (B $\in P$) and then show that **MAXFLOW** → B

Example of a Reduction: MAXFLOW → LP

- **Example**: Show that MAXFLOW is Poly-time that is, **MAXFLOW** $\in \boldsymbol{P}$
  - We need to find some Decision problem B that we already know is Poly-time (B $\in P$) and then show that **MAXFLOW** → B
  - Recall that we have shown that the **MAXFLOW** problem can be formulated as an **LP problem**.
  - The LP is a Decision problem that can be solved in Poly-time (by **Interior Point Methods** https://www.nytimes.com/1984/11/19/us/breakthrough-in-problem-solving.html – Not the Simplex method!)
  - So, we have the Reduction: **MAXFLOW → LP**, and hence **MAXFLOW** $\in \boldsymbol{P}$

Example of a Reduction: MAXFLOW → LP

- **MAXFLOW:**
  - <u>Input:</u> a directed graph $G = (V, E)$, $c_{ij} \in \mathbb{Q}$ cost on edge (i, j), start node S, end node T, $k \in \mathbb{Q}$
  - <u>Question:</u> is there a flow of value $\geq k$ from S to T that respects the edge costs & flow conservation constraints

?

→

- **LP:**
  - <u>Input:</u> $c \in \mathbb{Q}^{\mathrm{m}}$, $A \in \mathbb{Q}^{\mathrm{m} \times \mathrm{n}}$, $b \in \mathbb{Q}^{\mathrm{n}}$, $k \in \mathbb{Q}$
  - <u>Question:</u> is there an optimal value of the LP $\leq \mathrm{k}$

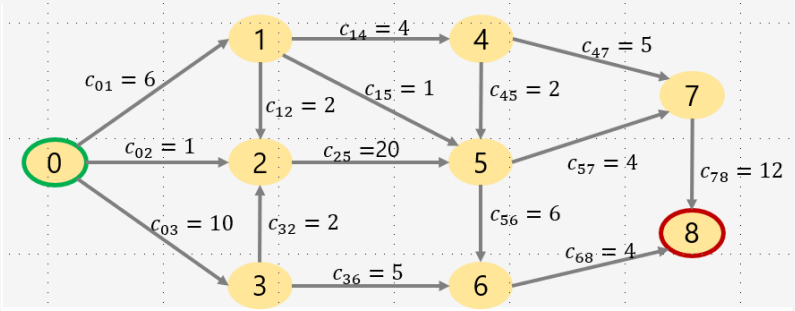
Example of a Reduction: MAXFLOW → LP

- **MAXFLOW:**
  - <u>Input:</u> a directed graph $G = (V, E)$, $c_{ij} \in \mathbb{Q}$ cost on edge (i, j), start node S, end node T, $k \in \mathbb{Q}$
  - <u>Question:</u> is there a flow of value $\geq k$ from S to T that respects the edge costs & flow conservation constraints

?

→

- **LP:**
  - <u>Input:</u> $c \in \mathbb{Q}^{m}$, $A \in \mathbb{Q}^{m \times n}$, $b \in \mathbb{Q}^{n}$, $k \in \mathbb{Q}$
  - <u>Question:</u> is there an optimal value of the LP $\leq$ k



$$
\begin{aligned}
\max \quad & f(x) = x_{01} + x_{02} + x_{03} \\
\text{s.t.} \quad & x_{01} = x_{14} + x_{15} + x_{12} \\
& x_{14} = x_{47} + x_{45} \\
& \dots \dots \dots \\
& 0 \leq x_{ij} \leq c_{ij}
\end{aligned}
$$

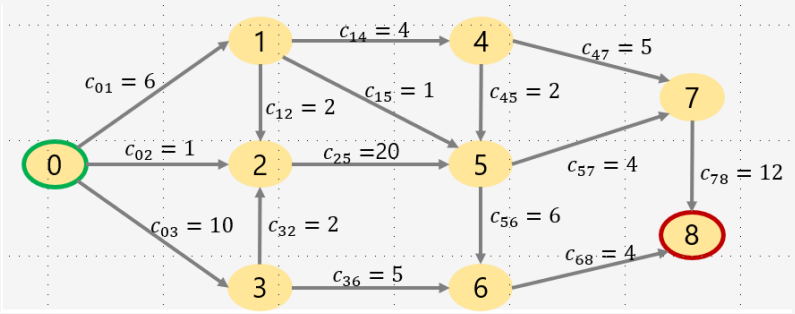
- We know how to model the **Maxflow problem** as a **Linear Programming problem**

Example of a Reduction: MAXFLOW → LP

- **MAXFLOW:**
  - <u>Input:</u> a directed graph $G = (V, E)$, $c_{ij} \in \mathbb{Q}$ cost on edge (i, j), start node S, end node T, $k \in \mathbb{Q}$
  - <u>Question:</u> is there a flow of value $\geq k$ from S to T that respects the edge costs & flow conservation constraints

→

- **LP:**
  - <u>Input:</u> $c \in \mathbb{Q}^{m}$, $A \in \mathbb{Q}^{m \times n}$, $b \in \mathbb{Q}^{n}$, $k \in \mathbb{Q}$
  - <u>Question:</u> is there an optimal value of the LP $\leq$ k



$$
\begin{aligned}
\max \quad & f(x) = x_{01} + x_{02} + x_{03} \\
\text{s.t.} \quad & x_{01} = x_{14} + x_{15} + x_{12} \\
& x_{14} = x_{47} + x_{45} \\
& \dots \dots \dots \\
& 0 \leq x_{ij} \leq c_{ij}
\end{aligned}
$$

- We know how to model the **Maxflow problem** as a **Linear Programming problem**
- We know that the **LP** $\in P$ $\Rightarrow$ Hence **MAXFLOW** $\in P$

Example of a Reduction: MAXFLOW → LP

- We also need to check that the Reduction from MAXFLOW to LP is **Polynomial in Length**
  - That is, the Linear Program is not too big
    - **does not have** exponential many decision variables or constraints,
    - **or data that takes** an exponential number of bits to write down and store on a computer.
  - The **size of the Linear Program** needs to be Polynomial in the size of the **instance of the MAXFLOW problem** (i.e., the original problem)
    - Without this requirement satisfied the Reduction is not possible.

MINCUT

- A **cut** is a <u>partition of the nodes of a graph</u> $G = (V, E)$ into <u>two non-empty disjoint sets</u> V_1 and V_2 (i.e., $V_1, V_2 \subseteq V$ and $V_1 \cup V_2 = V$, $V_1 \cap V_2 = \emptyset$)

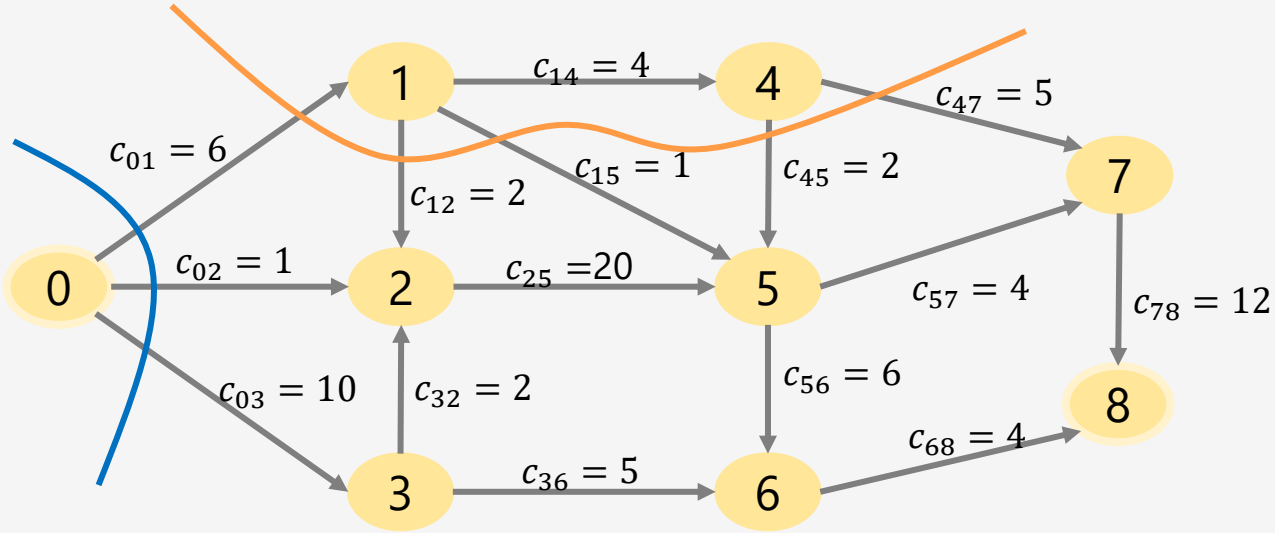
- The **value of a cut** is the sum of edge weights going from V_1 to V_2

Blue cut:

$V_1 = \{0\}$

$V_2 = \{1,2,\dots,8\}$

Value of blue cut=**??**



Orange cut:

$V_1 = \{1,4\}$

$V_2 = \{0,2,3,5,6,7,8\}$

Value of orange cut=**??**

MINCUT

- A **cut** is a partition of the nodes of a graph $G = (V, E)$ into two non-empty disjoint sets V_1 and V_2 (i.e., $V_1, V_2 \subseteq V$ and $V_1 \cup V_2 = V$, $V_1 \cap V_2 = \emptyset$)

- The **value of a cut** is the sum of edge weights going from V_1 to V_2

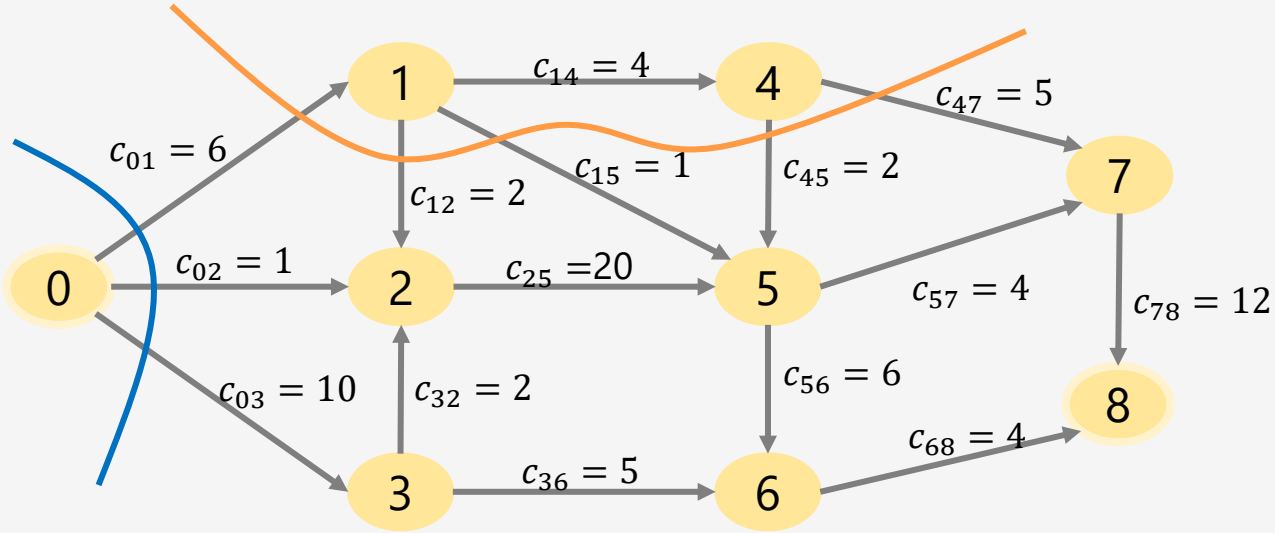
Blue cut:

$V_1 = \{0\}$

$V_2 = \{1,2,\dots,8\}$

Value of blue cut=

$6 + 1 + 10 = 17$



Orange cut:

$V_1 = \{1,4\}$

$V_2 = \{0,2,3,5,6,7,8\}$

Value of orange cut=

$2 + 1 + 2 + 5 = 10$

MINCUT

- The **MINCUT** Decision problem:
  - <u>Input:</u> a directed graph $G = (V, E)$, capacities $c_{ij} \in \mathbb{Q}$, $c_{ij} \geq 0$ for every edge, $k \in \mathbb{Q}$
  - <u>Question:</u> is there a cut of value $\leq k$?
- We want to check if: **MINCUT is** $\in \boldsymbol{P}$ **??**
- Any guess how to proceed ??
- What Poly-time Decision problem comes to our mind ??

MINCUT

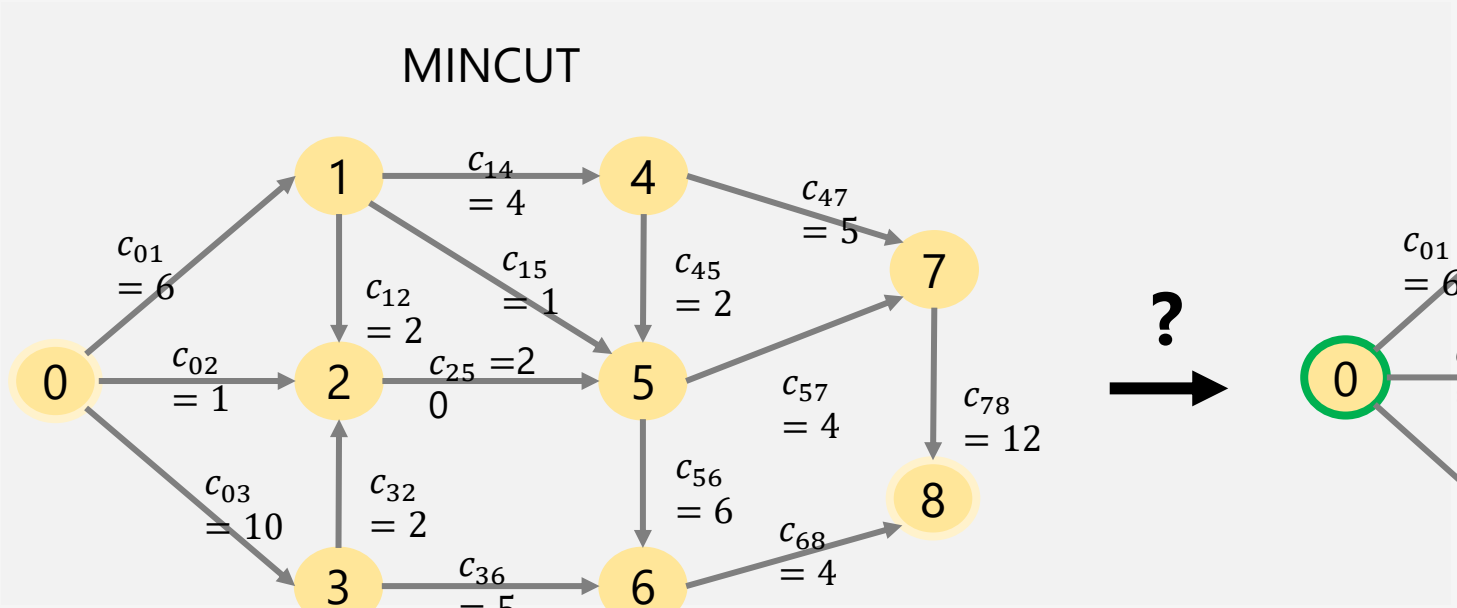
- The **MINCUT Decision problem**:
- <u>Input:</u> a directed graph $G = (V, E)$, capacities $c_{ij} \in \mathbb{Q}$, $c_{ij} \geq 0$ for every edge, $k \in \mathbb{Q}$
- <u>Question:</u> is there a cut of value $\leq k$?
- We want to **check if**: MINCUT is $\in P$??
- Any guess how to proceed ??
- What Poly-time Decision problem comes to our mind ??
- The **LP** Decision problem because $LP \in P$

MINCUT

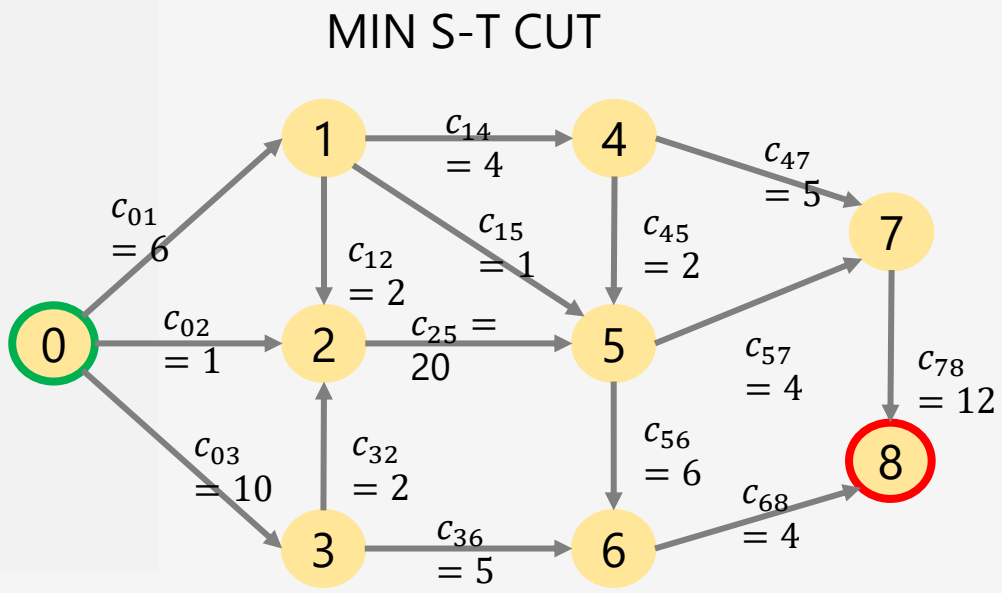
- There is no convenient way to formulate the general MINCUT problem as an LP.
- **But** we can easily formulate the **MIN S-T CUT Decision problem** as an **LP problem**
- **MIN S-T CUT**
  - Input: a directed graph $G = (V, E)$, capacities $c_{ij} \in \mathbb{Q}$, $c_{ij} \geq 0$ for every edge, $k \in \mathbb{Q}$, two designated nodes **S** and **T**
  - Question: is there a cut of value $\leq k$ that leaves **S** in one side and **T** in the other?
- **Strong Duality** allows us to formulate the **minimum S-T cut problem** as a maximum flow problem from node S to node T
- Hence: **MIN S-T CUT → MAXFLOW → LP.** Hence MIN S-T CUT $\in P$

MINCUT

- So far, we have shown **MIN S-T CUT** $\in \boldsymbol{P}$
- **But** we want to check whether **MINCUT** $\in \boldsymbol{P}$ **??**
- We have to find a **Poly-length Reduction** for MINCUT → MIN S-T CUT



?



MINCUT

- So far, we have shown **MIN S-T CUT** $\in \boldsymbol{P}$
- But we want to check whether **MINCUT** $\in \boldsymbol{P}$ **??**
- We have to find a **Poly-length Reduction** for MINCUT → MIN S-T CUT

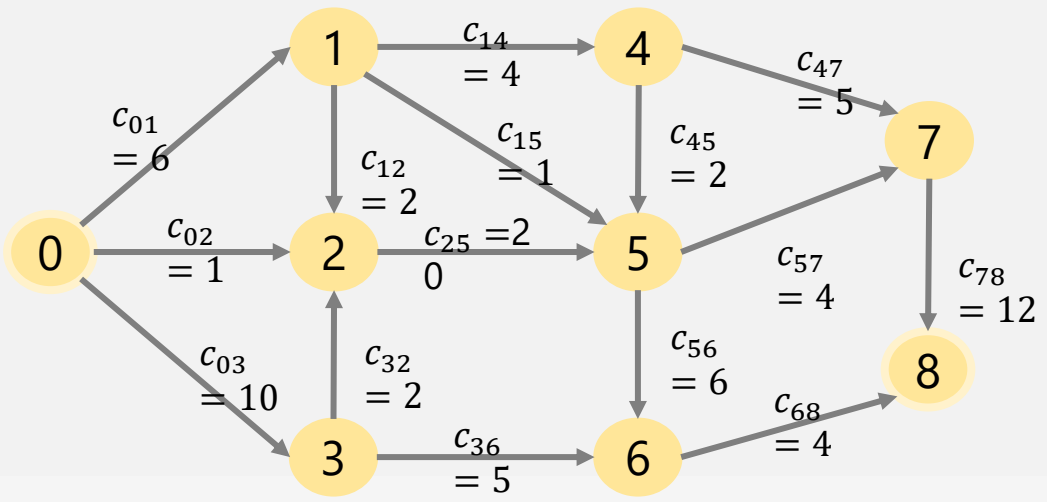
From the MINCUT **we can derive**

$$\binom{n}{2} = \frac{n!}{2!\,(n-2)!} \approx n^2$$

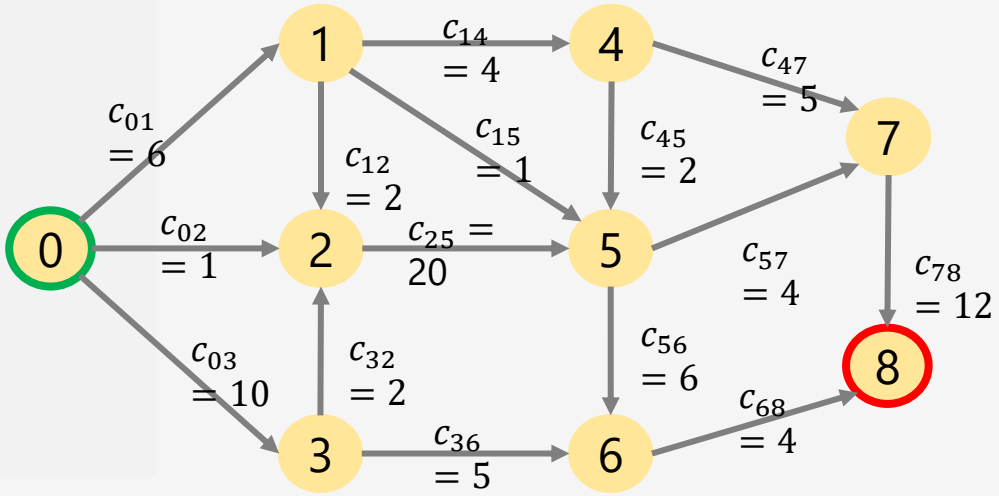
MIN S-T CUT problems.

It is a Poly-length Reduction

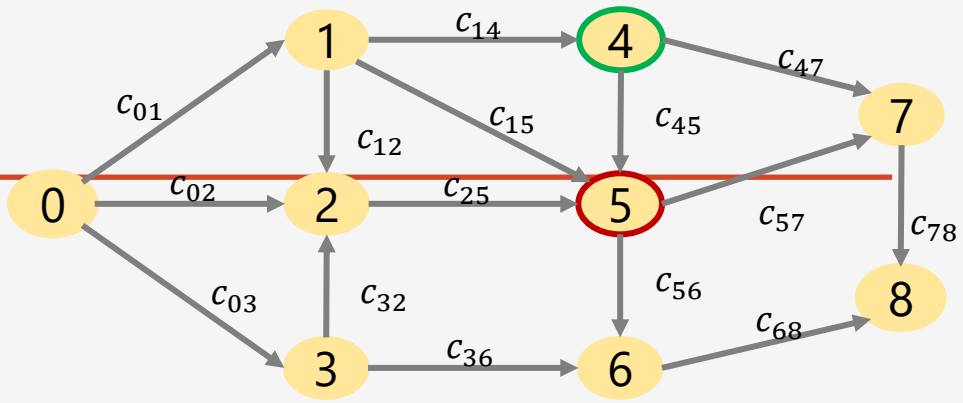
MINCUT



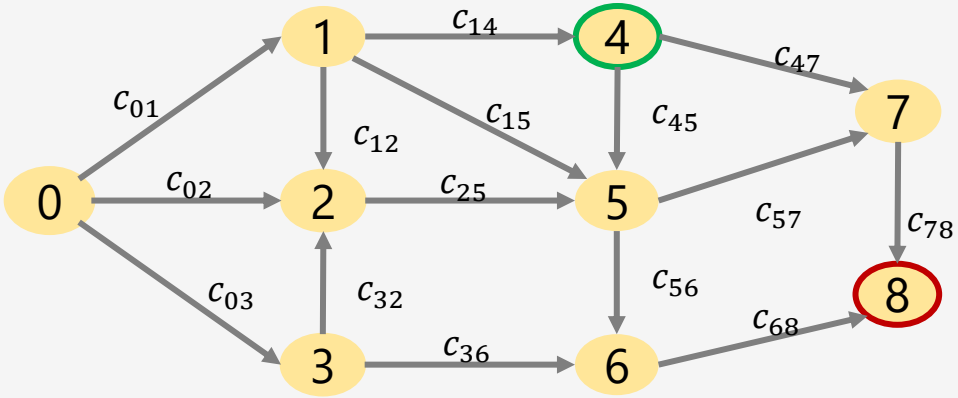
MIN S-T CUT

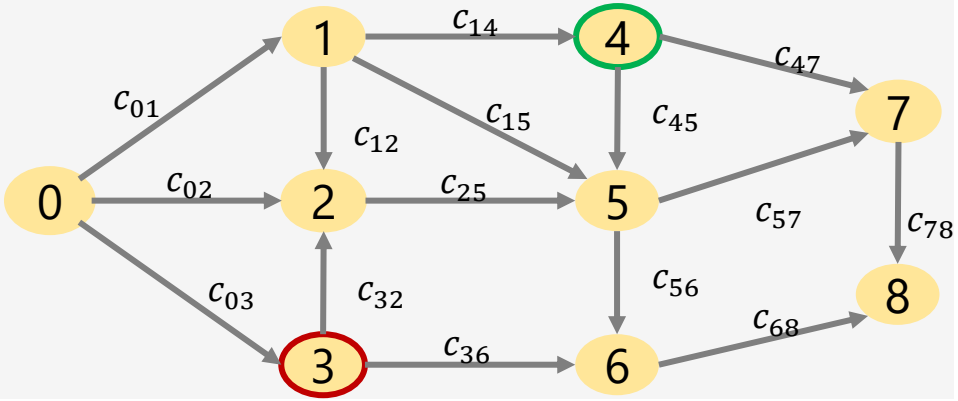


MINCUT



- **This is another Poly-length Reduction** for MINCUT → MIN S-T CUT
  - Compute MIN S-T CUT from a random node i to every other node
  - Compute MIN S-T CUT from every other node to the node i
  - Take the minimum over all the $\mathbf{2(|V|-1)}$ **cut values** you have computed
  - That minimum is the MINCUT





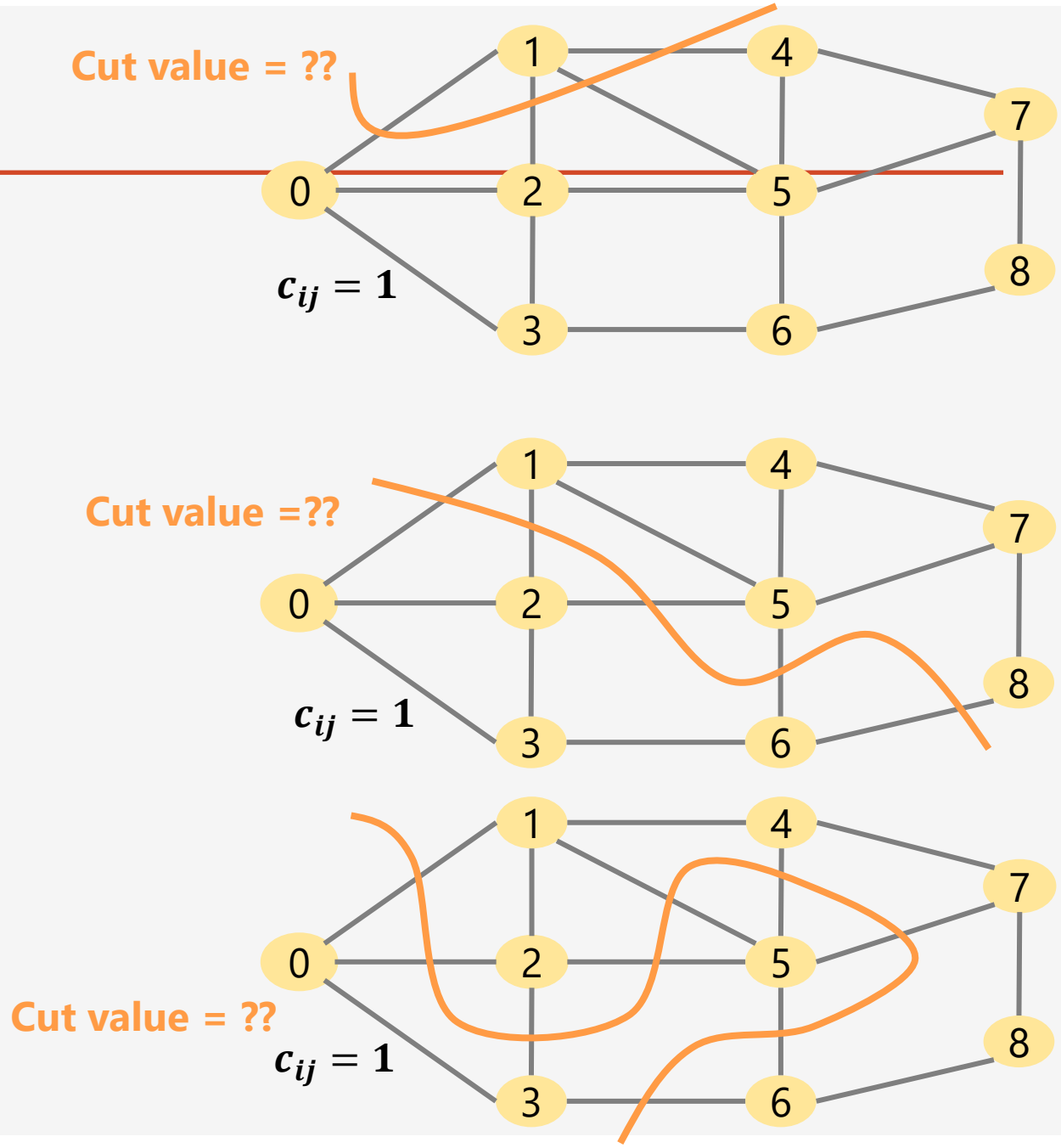
MINCUT

- Hence, we have shown the following Reductions:
  - MIN S-T CUT → MAXFLOW → LP
  - MINCUT → MIN S-T CUT

$\Rightarrow$ MINCUT → LP $\Rightarrow$ MINCUT $\in P$

MAXCUT

- The **MAXCUT** Decision problem:
- <u>Input:</u> an undirected graph $G = (V, E)$, capacities $c_{ij} \in \mathbb{Q}$, $c_{ij} \geq 0$ for every edge, $k \in \mathbb{Q}$
- <u>Question:</u> is there a cut of value $\geq k$?
- We want to check if: MAXCUT is $\in P$??

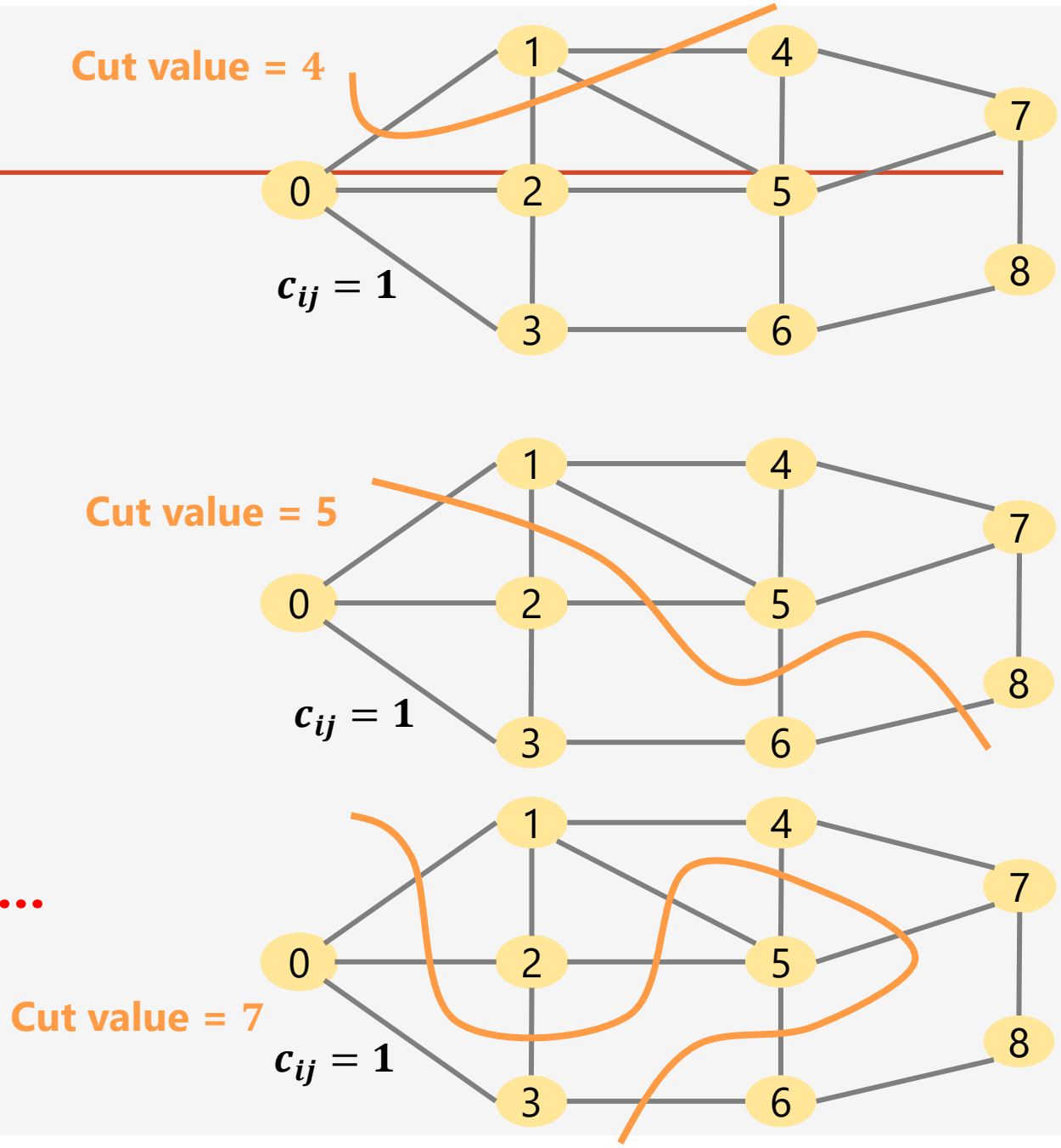


MAXCUT

- The **MAXCUT** Decision problem:
- <u>Input:</u> an undirected graph $G = (V, E)$, capacities $c_{ij} \in \mathbb{Q}$, $c_{ij} \geq 0$ for every edge, $k \in \mathbb{Q}$
- <u>Question:</u> is there a cut of value $\geq k$?
- We want to check if: MAXCUT is $\in P$??

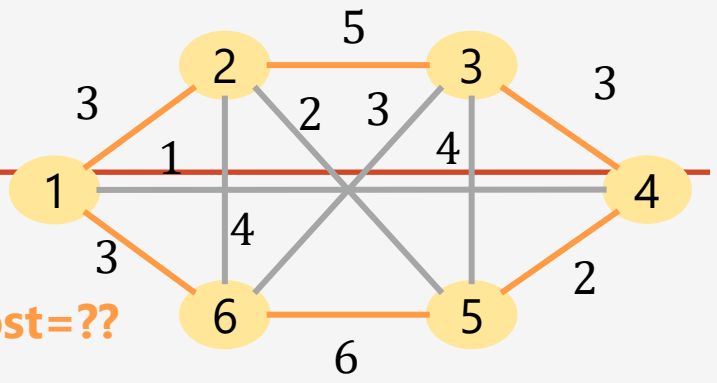
No Poly-time algorithm known so far…

<u>Note</u>: It is easy to **verify** if a cut has value $\geq k$, but **hard** to come up with the algorithm to compute it efficiently (Poly-time)

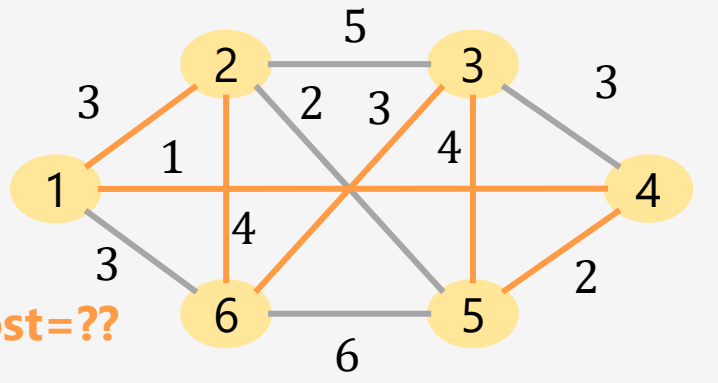


Traveling Salesman Problem (TSP)

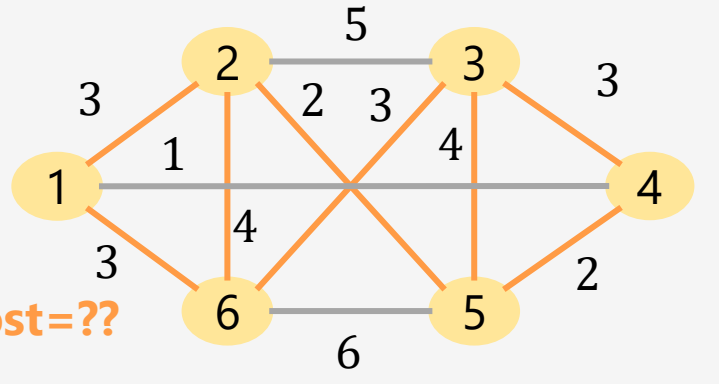
- The **TSP** Decision problem:
  - <u>Input:</u> an undirected graph $G = (V, E)$, traveling times $c_{ij} \in \mathbb{Q}$, $c_{ij} \geq 0$ for every edge, $k \in \mathbb{Q}$
  - <u>Question:</u> is there a **tour of cost** $\leq k$ that visits each node exactly once?
- We want to check if: **TSP is** $\in$ $\boldsymbol{P}$ **??**



tour of cost=??



tour of cost=??

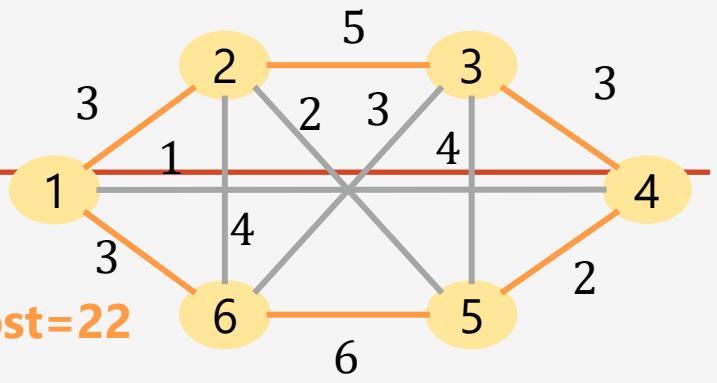


tour of cost=??

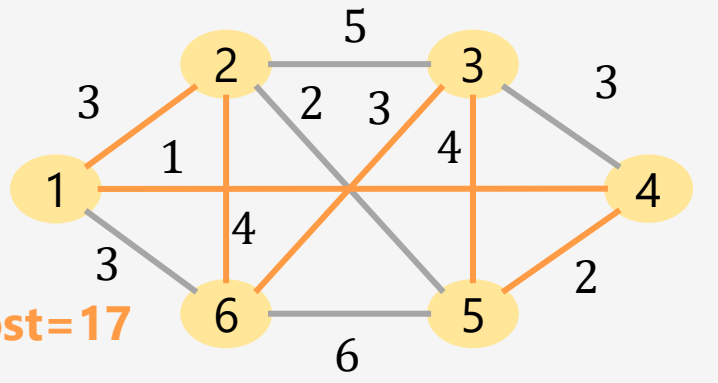
TSP

- The **TSP** Decision problem:
  - <u>Input:</u> an undirected graph $G = (V, E)$, traveling times $c_{ij} \in \mathbb{Q}$, $c_{ij} \geq 0$ for every edge, $k \in \mathbb{Q}$
  - <u>Question:</u> is there a **tour of cost** $\leq k$ that visits each node exactly once?
- We want to check if: TSP is $\in P$??
- **<u>No Poly-time algorithm known so far</u> …**
- There is Poly-time Reduction between MAXCUT and TSP. If you could show $MAXCUT \in P$ you have $TSP \in P$

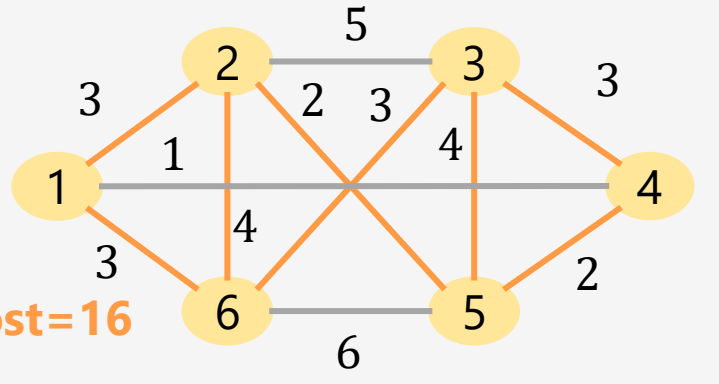
$1,000,000



tour of cost=22



tour of cost=17



tour of cost=16

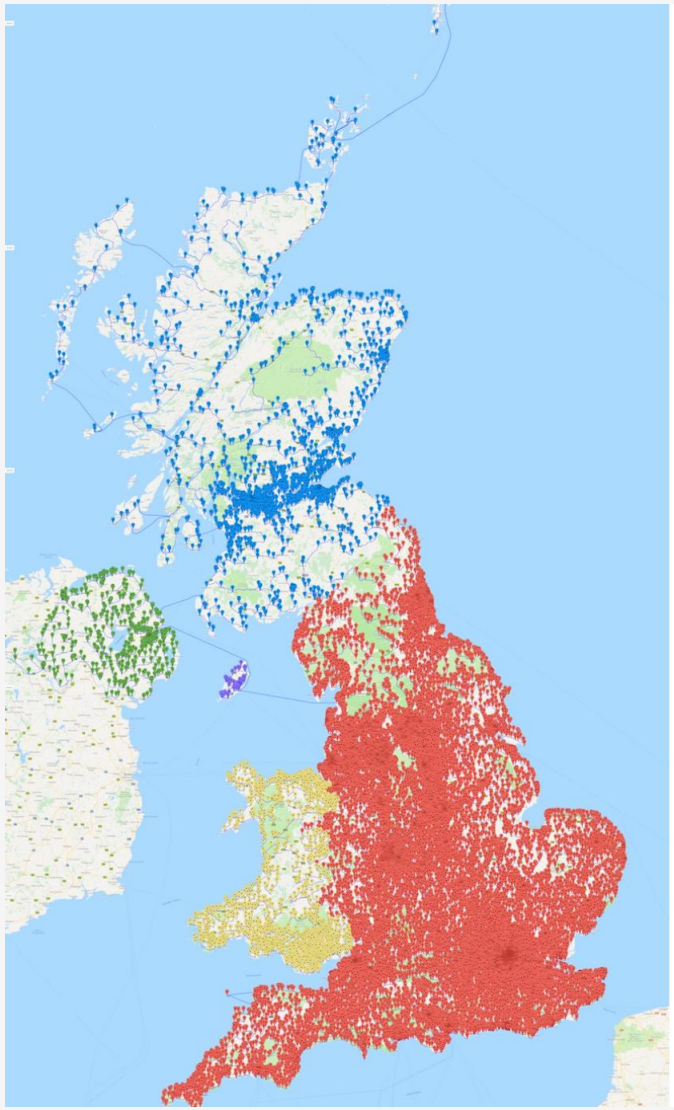
TSP

The TSP problem for all **Pubs in the UK**

- There are 49,687 Pubs

- Check more at:

https://www.math.uwaterloo.ca/tsp/uk/index.html

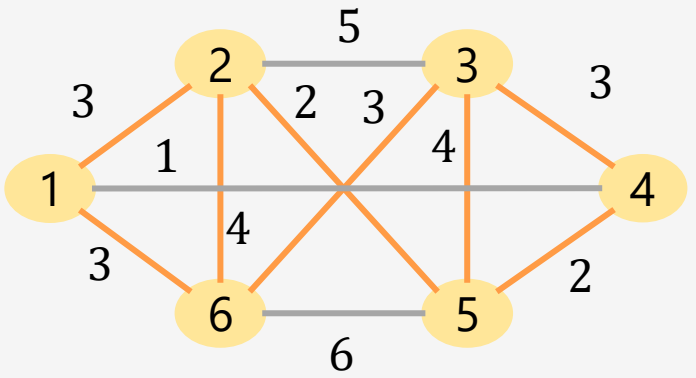


The complexity class NP

- A Decision problem belongs to the class NP if **the YES answer to any instance is <u>easily verifiable</u>.**
  - **In other words:** every YES instance has a **certificate of its correctness** that can be **verified in Poly-time as a function of the input size.**
- NP stands for **Nondeterministic Polynomial** time (NP does not mean Non-Polynomial)
- **Examples**:
  - LP, IP, PRIMES, TSP, MAXCUT, PENONPAPER, … **(Question:** Why PENONPAPER is in NP ??**)**
  - What is the **certificate** in each case?? (we will see soon)

The complexity class NP

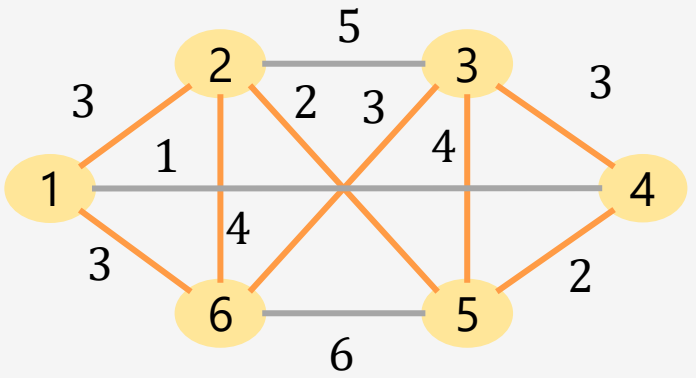
- **Example**: Let's see if the **TSP** Decision problem is in the NP class. Consider the instance on the right, with $k = 16$
  - If I tell you that the orange tour has cost 16
    - The orange tour the certificate
  - Then you can **easily verify** it… **HOW** ??



tour of cost=16

The complexity class NP

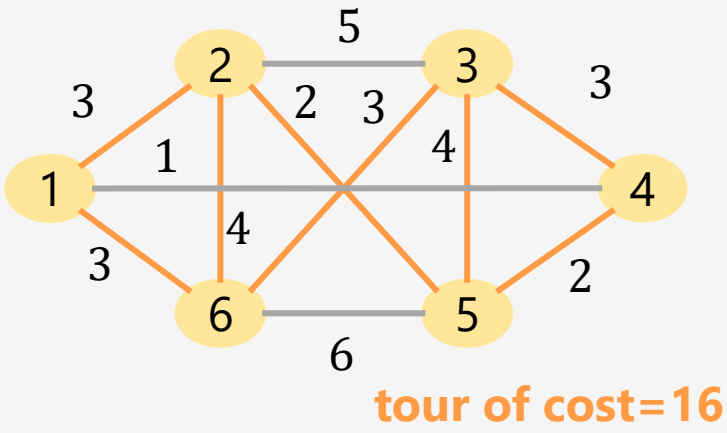
- **Example**: Let's see if the TSP Decision problem is in the NP class. Consider the instance on the right, with $k = 16$
  - If I tell you that the orange tour has cost 16
    - The orange tour the certificate
  - Then you can **easily verify** it… **HOW** ??
  - By adding the weights of the orange edges



tour of cost=16

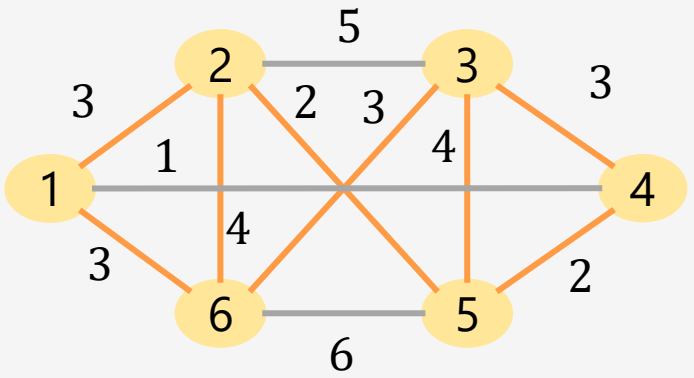
The complexity class NP

- **Example**: Let's see if the TSP Decision problem is in the NP class. Consider the instance on the right, with $k = 16$
  - If I tell you that the orange tour has cost 16
    - The orange tour the certificate
  - Then you can **easily verify** it… **HOW** ??
  - By adding the weights of the orange edges
  - But this process (the addition of the orange edge weights) can be performed in **Poly-time** **WHY** ??



The complexity class NP

- **Example**: Let's see if the TSP Decision problem is in the NP class. Consider the instance on the right, with $k = 16$



tour of cost=16

  - If I tell you that the orange tour has cost 16
    - The orange tour the certificate
  - Then you can **easily verify** it… **HOW** ??
  - By adding the weights of the orange edges
  - But this process (the addition of the orange edge weights) can be performed in **Poly-time** **WHY** ??
    - Because we can have a polynomial number of such edges w.r.t. nodes: $\frac{n(n-1)}{2}$

The complexity class NP

- A **Nondeterministic Computer** is a machine that can "guess" an answer and then verify it.
- NP class contains all Poly-time Decision problems, that is: $P \subseteq NP$ **WHY ??**
- NP class **also** contains many "difficult/hard" Decision problems
- **Examples** of problems in NP class:
  - LP, MAXFLOW, MINCUT, SDP, PRIMES, PENONPAPER, … (all $\in P$)
  - IP, TSP, MAXCUT, STABLE SET, SAT, 3SAT, KNAPSACK, COLORING, … (all other "difficult/hard" problems…)

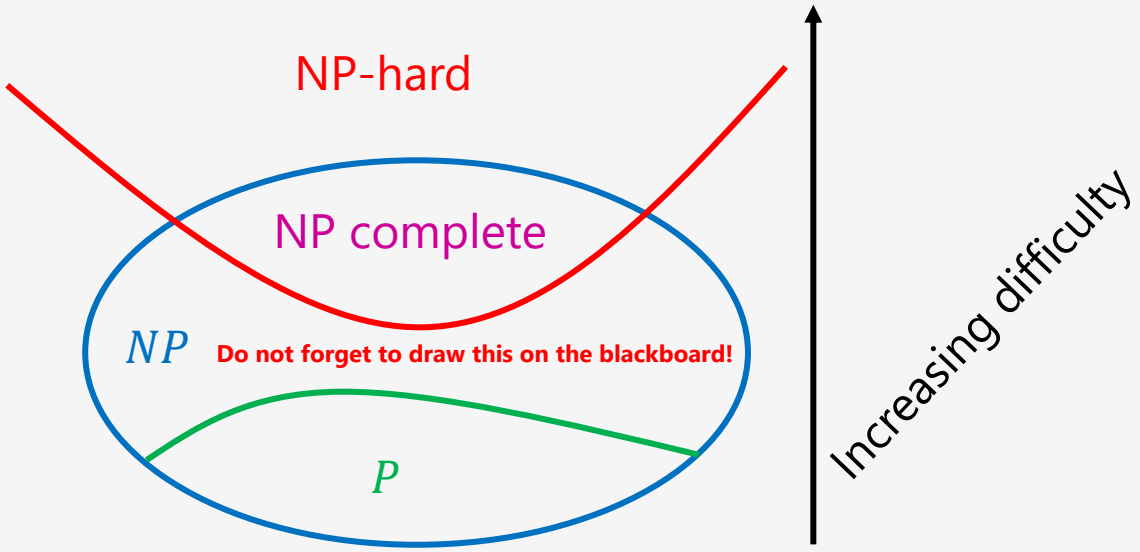
NP-hard & NP-complete problems

- A Decision problem is called NP-hard **iff** every problem instance in NP **reduces** to it via a Poly-time Reduction.
  (hardest than all problems in NP)

- A Decision problem is called NP complete iff

  1. It is NP hard
  2. It is in NP

  } The hardest problems in NP



- **Remarks**
  - NP hardness is shown by Reduction from a problem that is known to be NP hard
  - Membership in NP class is shown by presenting an easily checkable certificate of YES answer
  - NP hard problems may not be in NP (or may not be known to be in NP)

Reduction for NP-hard problems

- To show that a Decision problem is NP-hard we use Reductions (differently than in P…)
  - If A can be reduced to B ($A \rightarrow B$), and A is known to be NP-hard, **then B is NP-hard**

Reduction for NP-hard problems

- To show that a Decision problem is NP-hard we use Reductions (differently than in P…)
  - If A can be reduced to B ($A \rightarrow B$), and A is known to be NP-hard, **then B is NP-hard**
- Do you remember how we used Reduction process to show when a Decision problem is in P ??

Reduction for NP-hard problems

- To show that a Decision problem is NP-hard we use Reductions (differently than in P…)
  - If A can be reduced to B ($A \rightarrow B$), and A is known to be NP-hard, **then B is NP-hard**
- Do you remember how we used Reduction process to show when a Decision problem is in P ??
  - If A can be reduced to B ($A \rightarrow B$), and B is known to be $\in P$, **then $A \in P$.**
  - Do you see the difference in the use of Reductions for P and NP-hard ??

NP hard & NP complete problems

- Examples of NP-Complete Decision Problems
  - IP, TSP, MAXCUT, STABLESET, SAT, 3SAT, PARTITION, KNAPSACK, COLORING, VERTEXCOVER, SUDOKU
- Examples of Poly-time Decision Problems
  - ADDITION, MULTIPLICATION, LINEQ, LP, MAZFLOW, MINCUT, MATRIXPOS, SHORTEST PATH, SDPϵ, PRIMES, ZEROSUMNASH, PENONPAPER, …

Break point #1

Let’s take a moment to take a breath (Complexity is Complex for our minds…)

The Satisfiability problem – SAT

- SAT

One of the most fundamental NP-complete problems. Consider this to be known and use it in exercises.

  - Input: a Boolean formula in Conjunctive Normal Form (CNF)

  - Question: is there a 0/1 assignment to the variables that satisfy the formula?

$$\phi = \overbrace{(x \vee y \vee z) \wedge (x \vee \bar{y}) \wedge (y \vee \bar{z}) \wedge \underbrace{(\bar{x} \vee \bar{y} \vee \bar{z})}_{\textbf{clause}}}^{\textbf{Formula in CNF}}$$

Variables: x, y, z $\wedge$: AND, $\vee$: OR.

$\bar{x}$: NOT x (the **complement** of x)

Literal: a variable or its complement

$A \wedge B = C$

A	B	C
0	0	0
0	1	0
1	0	0
1	1	1

$A \vee B = C$

A	B	C
0	0	0
0	1	1
1	0	1
1	1	1

The Satisfiability problem – SAT

- **Examples**: Are the following formulas Satisfiable ??

YES or NO?? $\phi_1 = (x \vee y \vee z) \wedge (x \vee \bar{y}) \wedge (y \vee \bar{z}) \wedge (\bar{x} \vee \bar{y} \vee \bar{z})$

YES or NO?? $\phi_2 = (x \vee y \vee z) \wedge (x \vee \bar{y}) \wedge (y \vee \bar{z}) \wedge (\bar{x} \vee \bar{y} \vee \bar{z}) \wedge (\bar{x} \vee z) = \phi_1 \wedge (\bar{x} \vee z)$

The Satisfiability problem – SAT

- **Examples**: Are the following formulas Satisfiable ??

YES or NO?? $\phi_1 = (x \vee y \vee z) \wedge (x \vee \bar{y}) \wedge (y \vee \bar{z}) \wedge (\bar{x} \vee \bar{y} \vee \bar{z})$

YES: $(x, y, z) = (1,1,0) = (true, true, false)$

YES or NO?? $\phi_2 = (x \vee y \vee z) \wedge (x \vee \bar{y}) \wedge (y \vee \bar{z}) \wedge (\bar{x} \vee \bar{y} \vee \bar{z}) \wedge (\bar{x} \vee z) = \phi_1 \wedge (\bar{x} \vee z)$

NO

The Satisfiability problem – SAT

- **Examples**: Are the following formulas Satisfiable ??

YES or NO?? $\phi_1 = (x \vee y \vee z) \wedge (x \vee \bar{y}) \wedge (y \vee \bar{z}) \wedge (\bar{x} \vee \bar{y} \vee \bar{z})$

YES: $(x, y, z) = (1,1,0) = (true, true, false)$

SAT $\in NP$ because it is easy (Poly-time) to satisfy a YES answer

YES or NO?? $\phi_2 = (x \vee y \vee z) \wedge (x \vee \bar{y}) \wedge (y \vee \bar{z}) \wedge (\bar{x} \vee \bar{y} \vee \bar{z}) \wedge (\bar{x} \vee z) = \phi_1 \wedge (\bar{x} \vee z)$

NO

3SAT

- 3SAT
  - Input: a Boolean formula in Conjunctive Normal Form (CNF), where each clause has **exactly 3 literals**
  - Question: is there a 0/1 assignment to the variables that satisfy the formula?

$$\phi = (x \vee \bar{y} \vee z) \wedge (\bar{x} \vee y \vee \bar{z}) \wedge (x \vee w \vee \bar{z}) \wedge (\bar{y} \vee w \vee z)$$

- There is a simple Reduction from SAT to 3SAT that is: SAT → 3SAT [see: DPV, Chapter 8]
- Since we know that **SAT** is NP-hard, then **3SAT** is also NP-hard.
- We also know that **3SAT** is in NP. Hence **3SAT** is NP-complete

STABLE SET is NP-complete

- **Theorem:** Show that STABLE SET is NP-complete
- **Proof:** We need to show

1. STABLE SET $\in NP$

   ✓ YES, because if someone gives us k nodes that form a stable set, then we can easily check if they are connected

2. STABLE SET IS NP-hard

   ➢ We will prove this using the Reduction: 3SAT → STABLE SET

   - Remember that we know that 3SAT is NP-hard

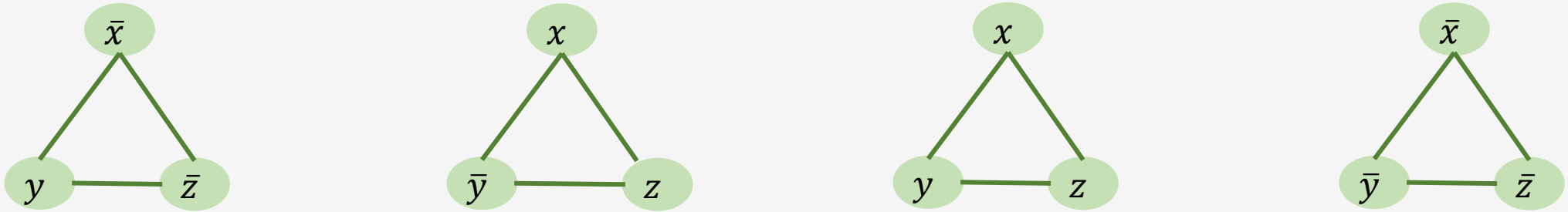
STABLE SET is NP-complete

- We will make the Reduction in 2 steps:
  - **Step 1:** for each clause in ϕ create a triangle
  - **Step 2:** across triangles we will connect each variable with its complement.
- This Reduction process will create a Graph

STABLE SET is NP-complete

- We use a random 3SAT instance: $\phi = (\bar{x} \vee y \vee \bar{z}) \wedge (x \vee \bar{y} \vee z) \wedge (x \vee y \vee z) \wedge (\bar{x} \vee \bar{y} \vee \bar{z})$

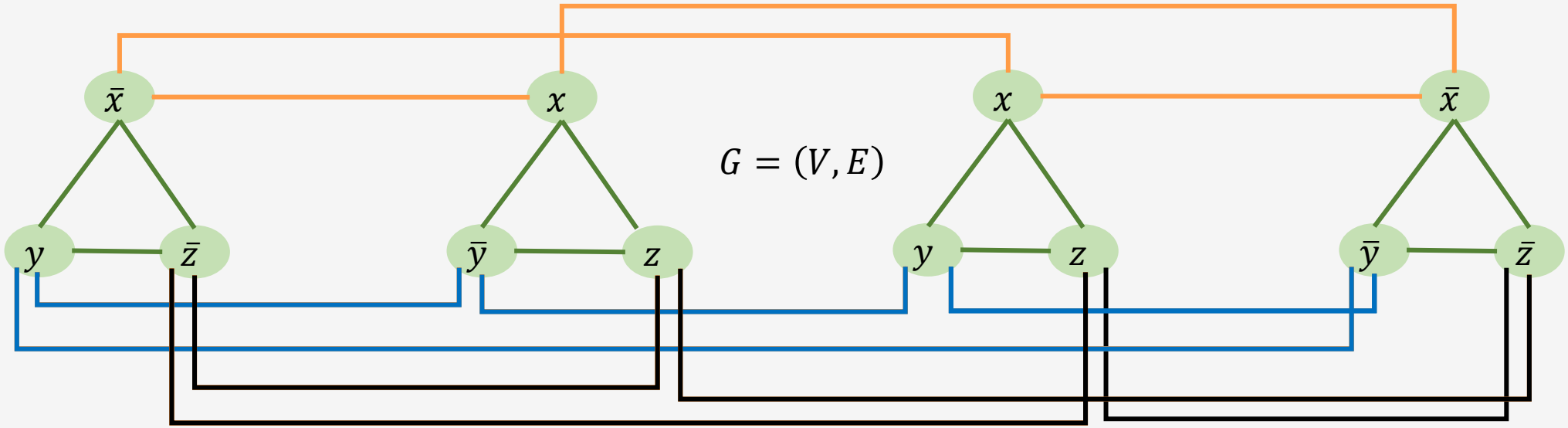
- Step 1:



STABLE SET is NP-complete

- We use a random 3SAT instance: $\phi = (\bar{x} \vee y \vee \bar{z}) \wedge (x \vee \bar{y} \vee z) \wedge (x \vee y \vee z) \wedge (\bar{x} \vee \bar{y} \vee \bar{z})$

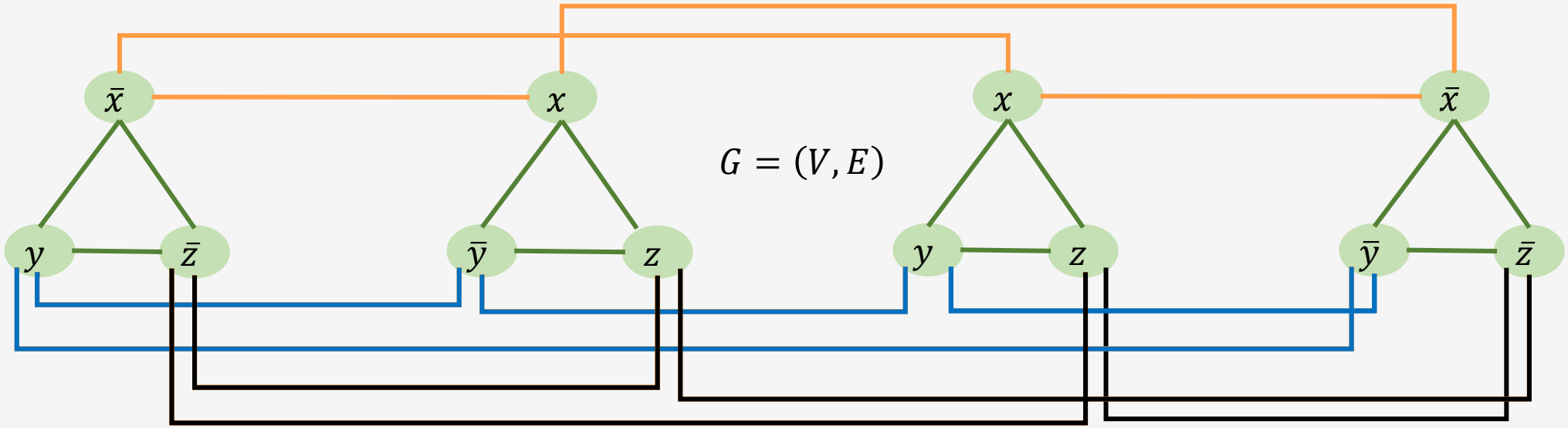
- Step 2:



STABLE SET is NP-complete

- We use a random 3SAT instance: $\phi = (\bar{x} \vee y \vee \bar{z}) \wedge (x \vee \bar{y} \vee z) \wedge (x \vee y \vee z) \wedge (\bar{x} \vee \bar{y} \vee \bar{z})$

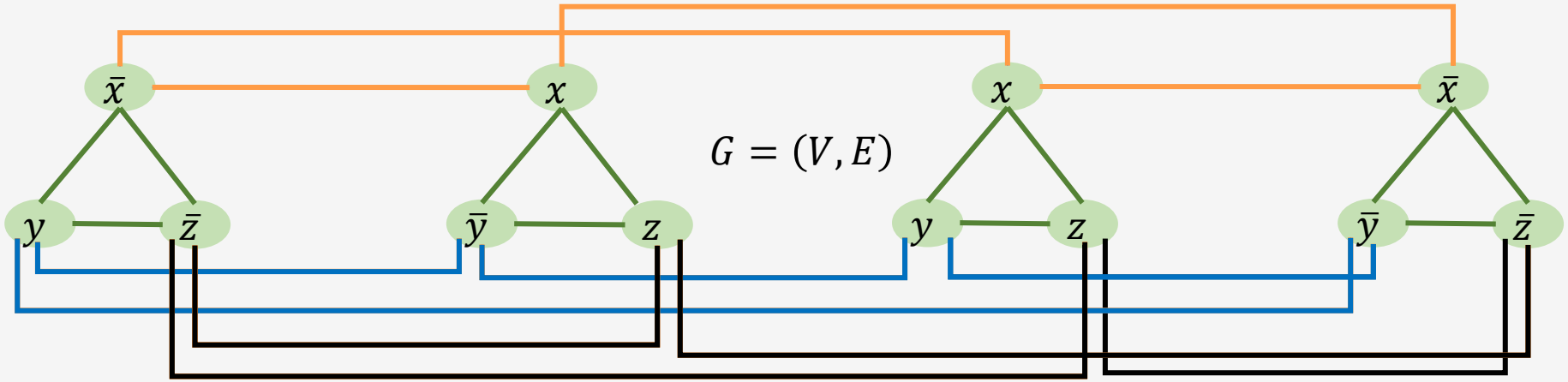
- Step 2:



The reduction 3SAT→STABLE SET is Poly-length;
That is, it does not produce exponential many nodes or edges in terms of the clauses in ϕ

STABLE SET is $\boldsymbol{NP}$-complete

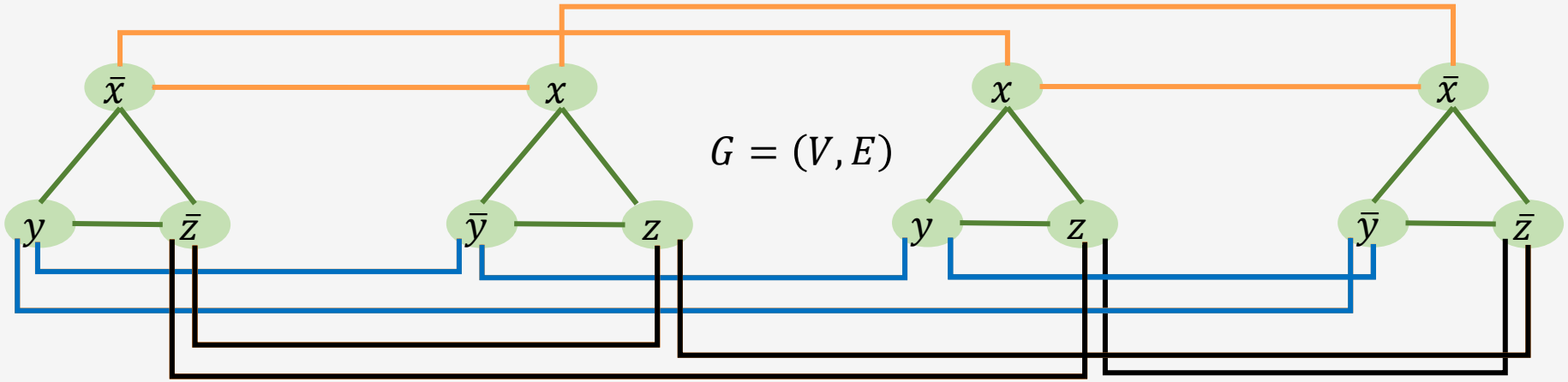
$$\phi = (\bar{x} \vee y \vee \bar{z}) \wedge (x \vee \bar{y} \vee z) \wedge (x \vee y \vee z) \wedge (\bar{x} \vee \bar{y} \vee \bar{z})$$



- We must prove the **claim**: ϕ is satisfiable $\Leftrightarrow \alpha(G) \geq k$ (for our example $k = 4$, the **number clauses in $\boldsymbol{\phi}$**)

STABLE SET is $\boldsymbol{NP}$-complete

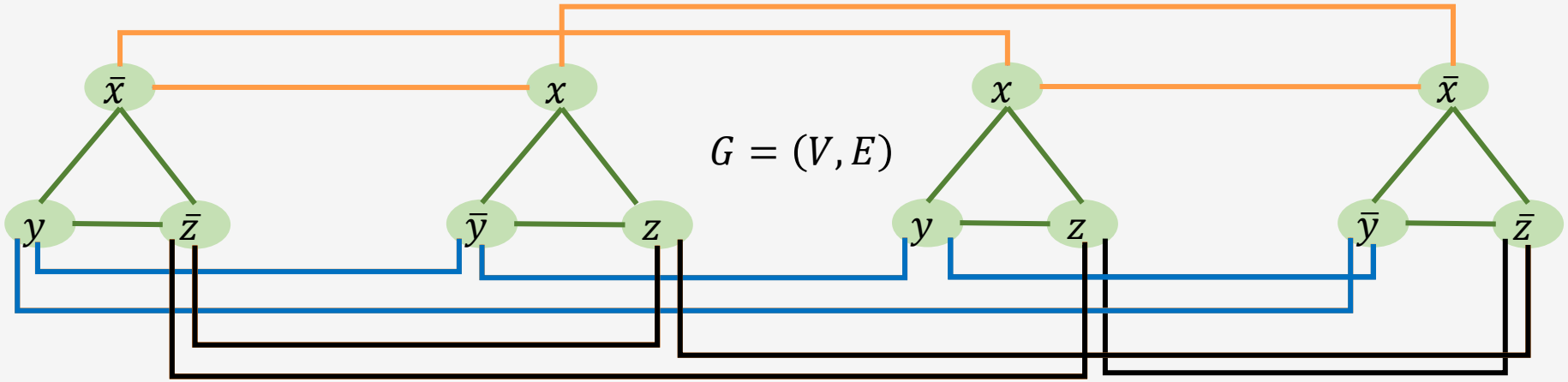
$$\phi = (\bar{x} \vee y \vee \bar{z}) \wedge (x \vee \bar{y} \vee z) \wedge (x \vee y \vee z) \wedge (\bar{x} \vee \bar{y} \vee \bar{z})$$



- We must prove the **claim**: ϕ is satisfiable $\Leftrightarrow$ $\alpha(G) \geq k$ (for our example $k = 4$, the number clauses in ϕ)
  - $(\Rightarrow)$ Suppose ϕ is satisfiable.
  - A possible $0/1$ assignment that makes $\phi = 1$ is: $(x, y, z) = (0,0,1)$

STABLE SET is NP-complete

$$\phi = (\bar{x} \vee y \vee \bar{z}) \wedge (x \vee \bar{y} \vee z) \wedge (x \vee y \vee z) \wedge (\bar{x} \vee \bar{y} \vee \bar{z})$$



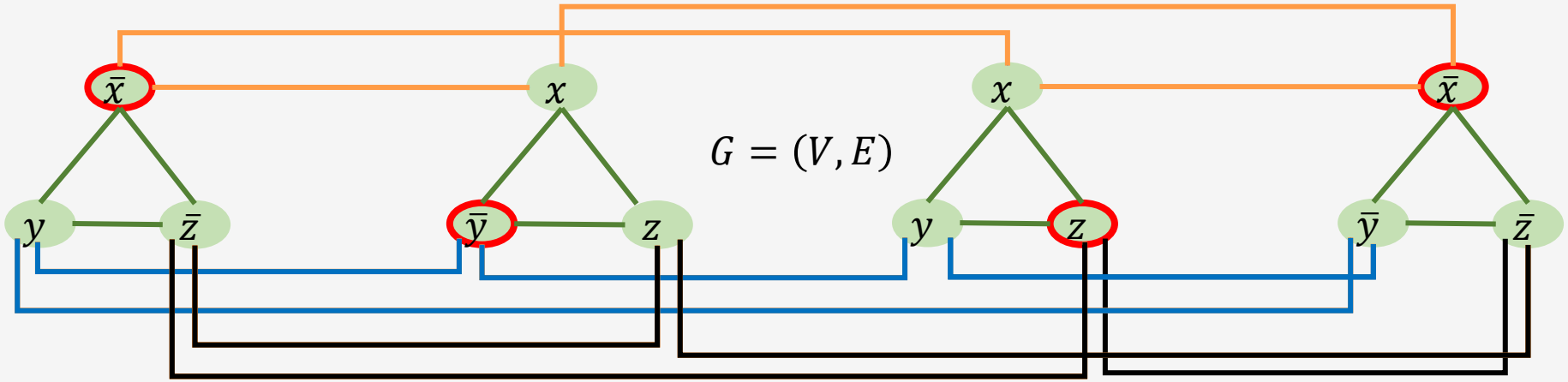
- We must prove the **claim**: ϕ is satisfiable $\Leftrightarrow \alpha(G) \geq k$ (for our example $k = 4$, the number clauses in ϕ)
  - $(\Rightarrow)$ Suppose ϕ is satisfiable.
  - A possible $0/1$ assignment that makes $\phi = 1$ is: $(x, y, z) = (0,0,1)$
  - In order to show that $\alpha(G) \geq 4$ we can create the following process:
    - **Whatever literal we set to 1 in a clause, we pick it in the stable set**

$$\phi = (\bar{x} \vee y \vee \bar{z}) \wedge (x \vee \bar{y} \vee z) \wedge (x \vee y \vee z) \wedge (\bar{x} \vee \bar{y} \vee \bar{z})$$

$$1 \quad 0 \quad 0 \qquad 0 \quad 1 \quad 1 \qquad 0 \quad 0 \quad 1 \qquad 1 \quad 1 \quad 0$$

STABLE SET is NP-complete

$$\phi = (\bar{x} \vee y \vee \bar{z}) \wedge (x \vee \bar{y} \vee z) \wedge (x \vee y \vee z) \wedge (\bar{x} \vee \bar{y} \vee \bar{z})$$



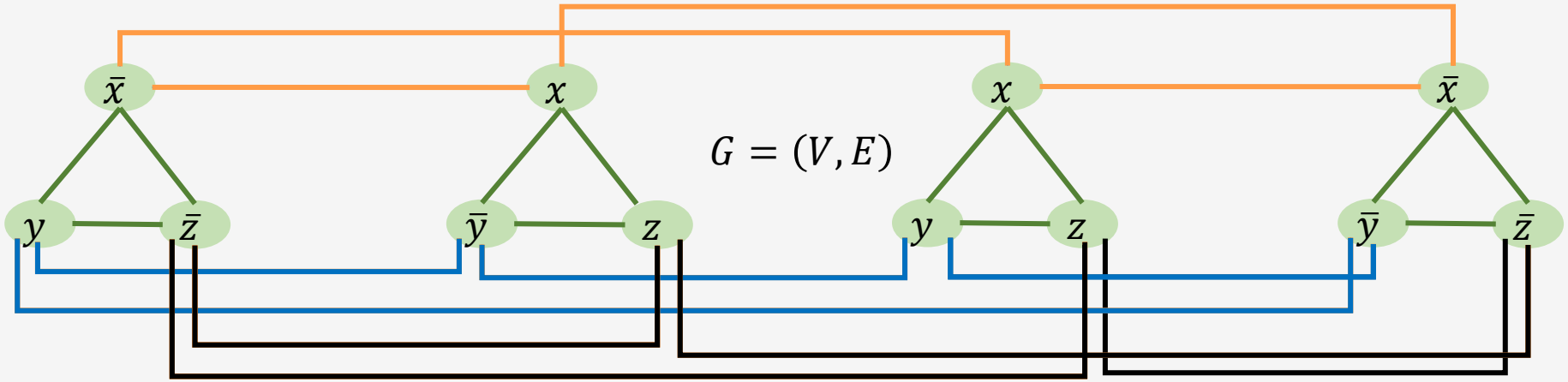
- We must prove the **claim**: ϕ is satisfiable $\Leftrightarrow \alpha(G) \geq k$ (for our example $k = 4$, the number clauses in ϕ)
  - $(\Rightarrow)$ Suppose ϕ is satisfiable.
  - A possible $0/1$ assignment that makes $\phi = 1$ is: $(x, y, z) = (0,0,1)$
  - In order to show that $\alpha(G) \geq 4$ we can create the following process:
    - **Whatever literal we set to 1 in a clause, we pick it in the stable set**

$$\phi = (\bar{x} \vee y \vee \bar{z}) \wedge (x \vee \bar{y} \vee z) \wedge (x \vee y \vee z) \wedge (\bar{x} \vee \bar{y} \vee \bar{z})$$

$$1 \quad 0 \quad 0 \qquad 0 \quad 1 \quad 1 \qquad 0 \quad 0 \quad 1 \qquad 1 \quad 1 \quad 0$$

STABLE SET is $\boldsymbol{NP}$-complete

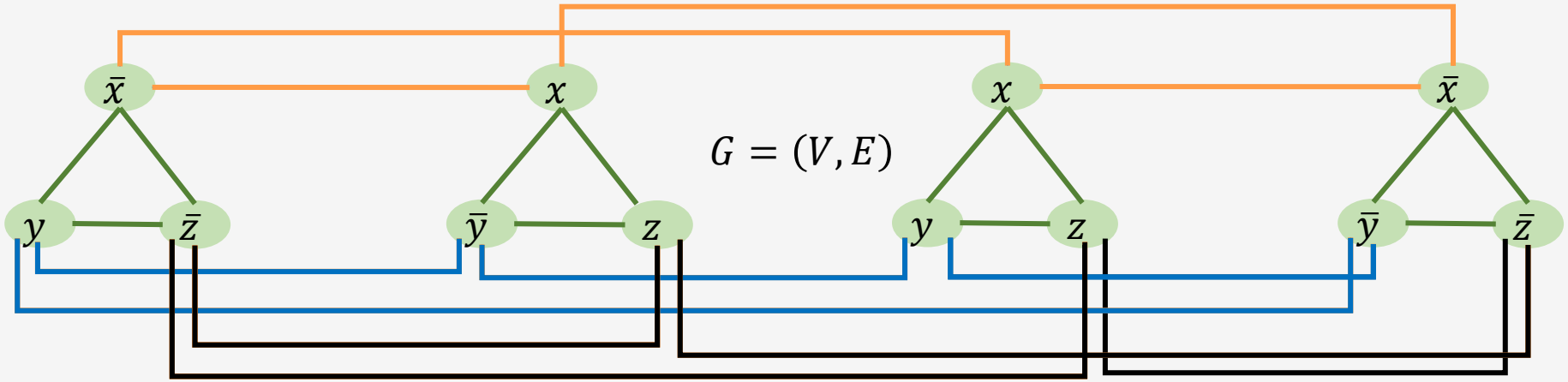
$$\phi = (\bar{x} \vee y \vee \bar{z}) \wedge (x \vee \bar{y} \vee z) \wedge (x \vee y \vee z) \wedge (\bar{x} \vee \bar{y} \vee \bar{z})$$



- We must prove the **claim**: ϕ is satisfiable $\Leftrightarrow \alpha(G) \geq k$ (for our example $k = 4$)
  - ($\Leftarrow$) Suppose $\alpha(G) \geq 4$.

STABLE SET is $\boldsymbol{NP}$-complete

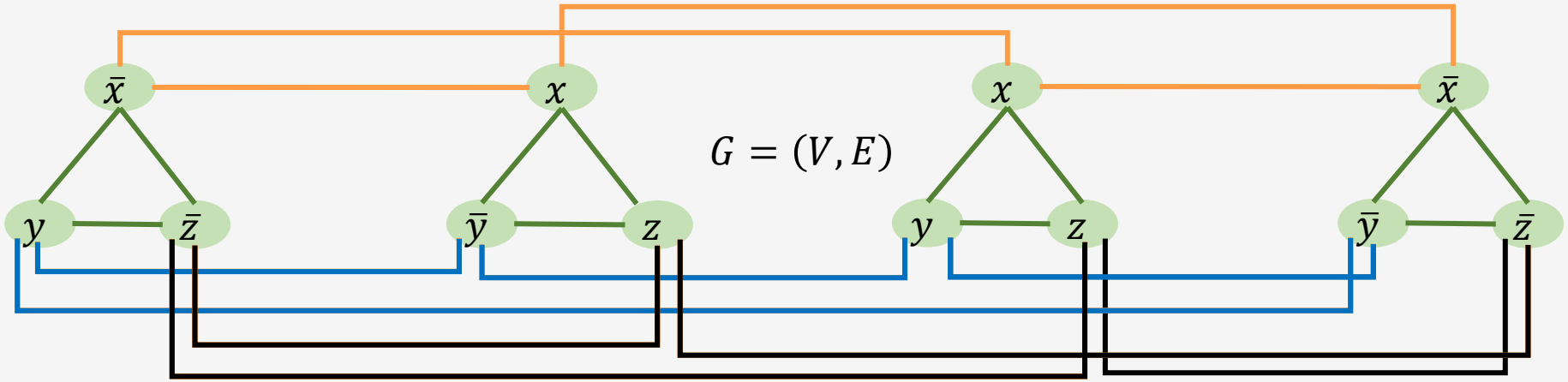
$$\phi = (\bar{x} \vee y \vee \bar{z}) \wedge (x \vee \bar{y} \vee z) \wedge (x \vee y \vee z) \wedge (\bar{x} \vee \bar{y} \vee \bar{z})$$



- We must prove the **claim**: ϕ is satisfiable $\Leftrightarrow \alpha(G) \geq k$ (for our example $k = 4$)
  - ($\Leftarrow$) Suppose $\alpha(G) \geq 4$.
  - This implies that $\alpha(G) = 4$ (WHY??)

STABLE SET is $\boldsymbol{NP}$-complete

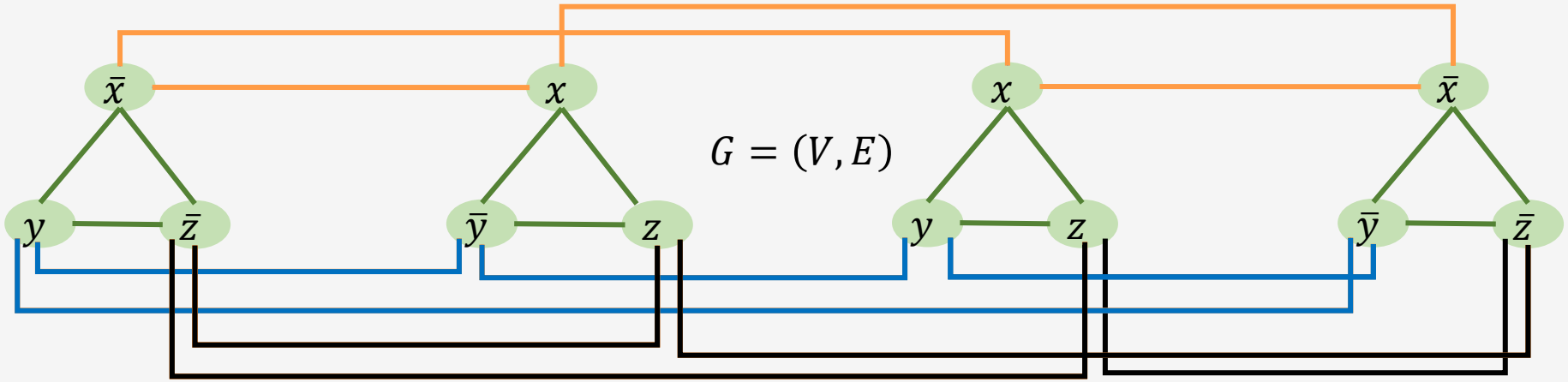
$$\phi = (\bar{x} \vee y \vee \bar{z}) \wedge (x \vee \bar{y} \vee z) \wedge (x \vee y \vee z) \wedge (\bar{x} \vee \bar{y} \vee \bar{z})$$



- We must prove the **claim**: ϕ is satisfiable $\Leftrightarrow \alpha(G) \geq k$ (for our example $k = 4$)
  - ($\Leftarrow$) Suppose $\alpha(G) \geq 4$.
  - This implies that $\alpha(G) = 4$ (WHY??) → because if we had $\alpha(G) = 5$, two nodes must be in one triangle…

STABLE SET is NP-complete

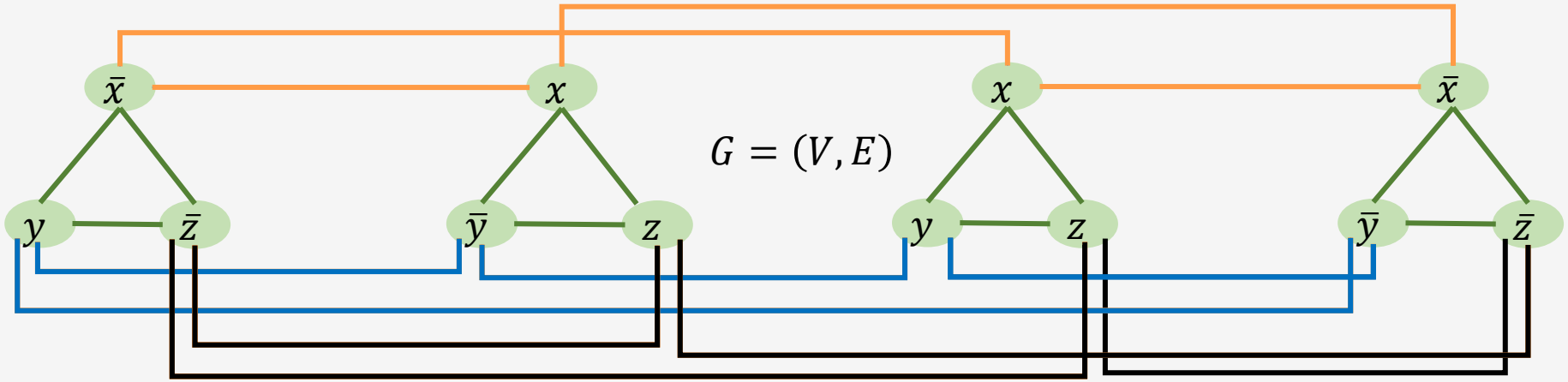
$$\phi = (\bar{x} \vee y \vee \bar{z}) \wedge (x \vee \bar{y} \vee z) \wedge (x \vee y \vee z) \wedge (\bar{x} \vee \bar{y} \vee \bar{z})$$



- We must prove the **claim**: ϕ is satisfiable $\Leftrightarrow \alpha(G) \geq k$ (for our example $k = 4$)
  - ($\Leftarrow$) Suppose $\alpha(G) \geq 4$.
  - This implies that $\alpha(G) = 4$ (WHY??) → because if we had $\alpha(G) = 5$, two nodes must be in one triangle…
  - Hence the stable set will have one node per triangle.

STABLE SET is NP-complete

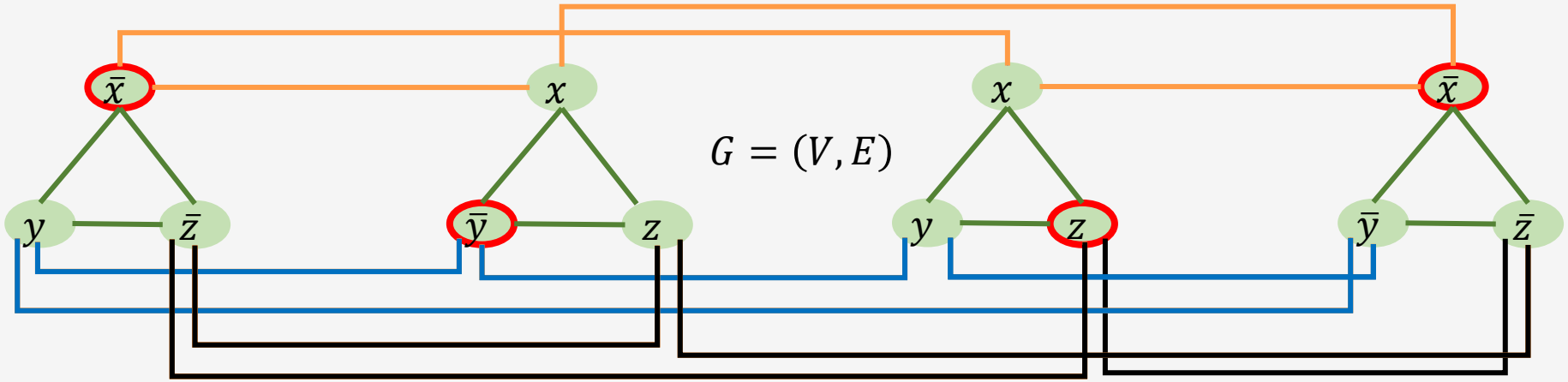
$$\phi = (\bar{x} \vee y \vee \bar{z}) \wedge (x \vee \bar{y} \vee z) \wedge (x \vee y \vee z) \wedge (\bar{x} \vee \bar{y} \vee \bar{z})$$



- We must prove the **claim**: ϕ is satisfiable $\Leftrightarrow \alpha(G) \geq k$ (for our example $k = 4$)
  - ($\Leftarrow$) Suppose $\alpha(G) \geq 4$.
  - This implies that $\alpha(G) = 4$ (WHY??) → because if we had $\alpha(G) = 5$, two nodes must be in one triangle…
  - Hence the stable set will have one node per triangle.
  - To find a $0/1$ assignment that makes $\phi = 1$, follow the process:
    - Select one node in each triangle

STABLE SET is NP-complete

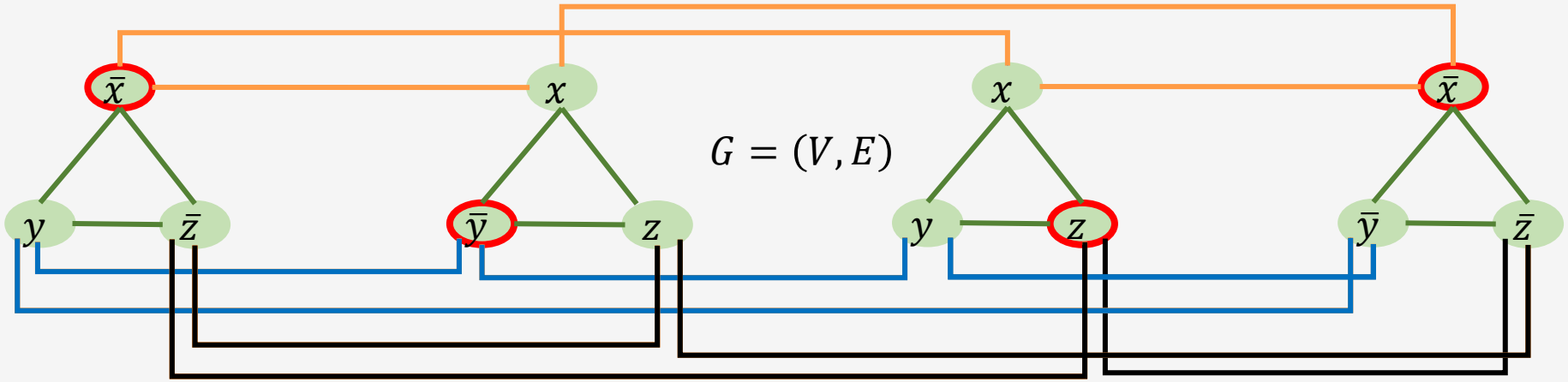
$$\phi = (\bar{x} \vee y \vee \bar{z}) \wedge (x \vee \bar{y} \vee z) \wedge (x \vee y \vee z) \wedge (\bar{x} \vee \bar{y} \vee \bar{z})$$



- We must prove the **claim**: ϕ is satisfiable $\Leftrightarrow \alpha(G) \geq k$ (for our example $k = 4$)
  - ($\Leftarrow$) Suppose $\alpha(G) \geq 4$.
  - This implies that $\alpha(G) = 4$ (WHY??) → because if we had $\alpha(G) = 5$, two nodes must be in one triangle…
  - Hence the stable set will have one node per triangle.
  - To find a $0/1$ assignment that makes $\phi = 1$, follow the process:
    - Select one node in each triangle

STABLE SET is NP-complete

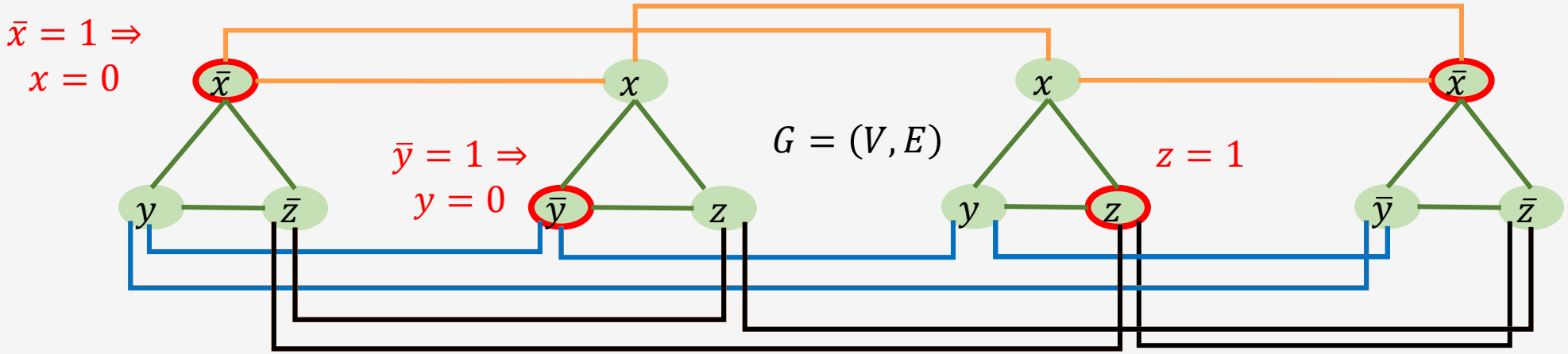
$$\phi = (\bar{x} \vee y \vee \bar{z}) \wedge (x \vee \bar{y} \vee z) \wedge (x \vee y \vee z) \wedge (\bar{x} \vee \bar{y} \vee \bar{z})$$



- We must prove the **claim**: ϕ is satisfiable $\Leftrightarrow \alpha(G) \geq k$ (for our example $k = 4$)
  - ($\Leftarrow$) Suppose $\alpha(G) \geq 4$.
  - This implies that $\alpha(G) = 4$ (WHY??) → because if we had $\alpha(G) = 5$, two nodes must be in one triangle…
  - Hence the stable set will have one node per triangle.
  - To find a $0/1$ assignment that makes $\phi = 1$, follow the process:
    - Select one node in each triangle
    - Assign 1 to the corresponding literal in ϕ

STABLE SET is NP-complete

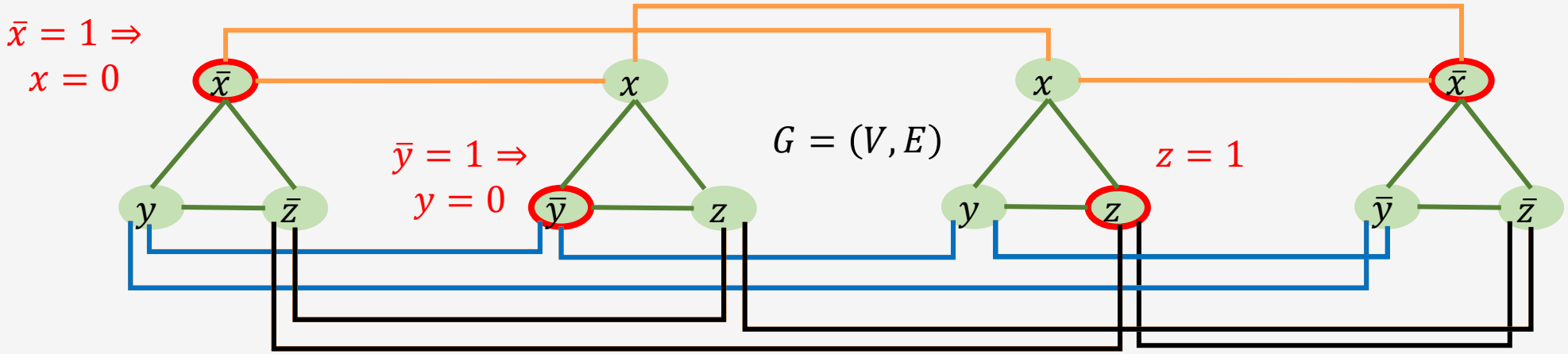
$$\phi = (\bar{x} \vee y \vee \bar{z}) \wedge (x \vee \bar{y} \vee z) \wedge (x \vee y \vee z) \wedge (\bar{x} \vee \bar{y} \vee \bar{z})$$



- We must prove the **claim**: ϕ is satisfiable $\Leftrightarrow \alpha(G) \geq k$ (for our example $k = 4$)
  - ($\Leftarrow$) Suppose $\alpha(G) \geq 4$.
  - This implies that $\alpha(G) = 4$ (WHY??) → because if we had $\alpha(G) = 5$, two nodes must be in one triangle…
  - Hence the stable set will have one node per triangle.
  - To find a $0/1$ assignment that makes $\phi = 1$, follow the process:
    - Select one node in each triangle
    - Assign 1 to the corresponding literal in ϕ

STABLE SET is NP-complete

$$\phi = (\bar{x} \vee y \vee \bar{z}) \wedge (x \vee \bar{y} \vee z) \wedge (x \vee y \vee z) \wedge (\bar{x} \vee \bar{y} \vee \bar{z})$$



- We must prove the **claim**: ϕ is satisfiable $\Leftrightarrow \alpha(G) \geq k$ (for our example $k = 4$)
  - ($\Leftarrow$) Suppose $\alpha(G) \geq 4$.
  - This implies that $\alpha(G) = 4$ (WHY??) → because if we had $\alpha(G) = 5$, two nodes must be in one triangle…
  - Hence the stable set will have one node per triangle.
  - To find a $0/1$ assignment that makes $\phi = 1$, follow the process:
    - Select one node in each triangle
    - Assign 1 to the corresponding literal in ϕ

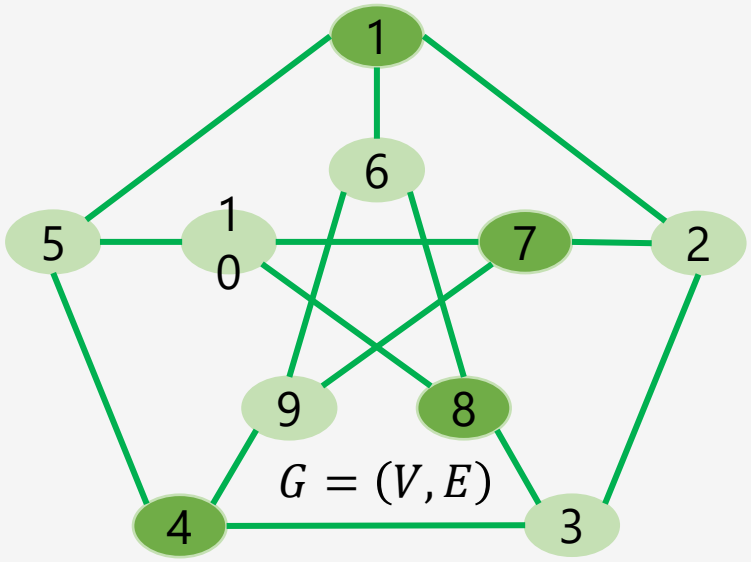
$$\phi = (\bar{x} \vee y \vee \bar{z}) \wedge (x \vee \bar{y} \vee z) \wedge (x \vee y \vee z) \wedge (\bar{x} \vee \bar{y} \vee \bar{z}) = 1$$

$$1 \quad 0 \quad 0 \qquad 0 \quad 1 \quad 1 \qquad 0 \quad 0 \quad 1 \qquad 1 \quad 1 \quad 0$$

IP is NP-complete

- **Theorem**: Show that IP is NP-complete
- **Proof**: Consider the Reduction:
  - STABLE SET → IP
  - STABLE SET is NP-complete
  - Hence IP is NP-complete

STABLE SET



$\boldsymbol{\alpha(G) \geq k}$

IP

- **Decision variables:**
  - $x_i = 1$, if node i is picked;
  - $x_i = 0$, otherwise
- **Objective:**
  - $\cancel{\max f(x) = \sum_{i=1}^{10} x_i}$
- **Constraints:**
  - $\boldsymbol{\sum_{i=1}^{10} x_i \geq k}$
  - $x_i + x_j \leq 1, \forall (i,j) \in E$
  - $x_i \in \{0,1\}, \forall i \in V$

Feasibility of Quadratic Equations is NP-complete

- **Theorem**: Show that the Feasibility Of Quadratic Equations (FQE) is NP-complete

  - FQE:
    - <u>Input:</u> Given $Q_i \in \mathbb{Q}^{n \times n}$ and $b_i, \in \mathbb{Q}^n$, for $i = 1, \dots m$, $c \in \mathbb{Q}^m$
    - <u>Question:</u> Is there a vector $x \in \mathbb{Q}^n$ that satisfies the equations: $\mathrm{x}^{\mathrm{T}} Q_i x + b_i^T x = c_i$ for all $i = 1, \dots, m$

Feasibility of Quadratic Equations is NP-complete

- **Theorem**: Show that the Feasibility Of Quadratic Equations (FQE) is NP-complete

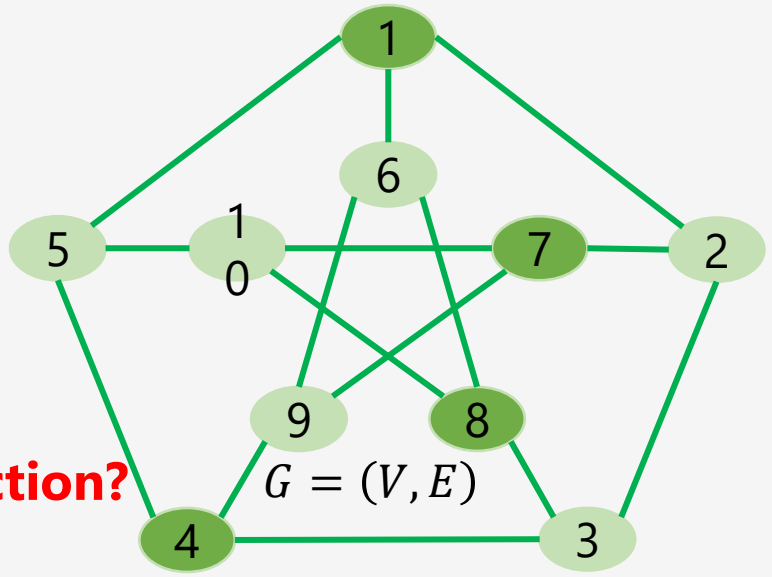
- **Proof**: We consider the Reduction:
  - STABLE SET → FQE
  - STABLE SET is NP-complete
  - Hence FQE is NP-complete

- FQE:
  - <u>Input:</u> Given $Q_i \in \mathbb{Q}^{n \times n}$ and $b_i, \in \mathbb{Q}^n$, for $i = 1, \dots m$, $c \in \mathbb{Q}^m$
  - <u>Question:</u> Is there a vector $x \in \mathbb{Q}^n$ that satisfies the equations: $\mathrm{x}^{\mathrm{T}} Q_i x + b_i^T x = c_i$ for all $i = 1, \dots, m$

Feasibility of Quadratic Equations is NP-complete

- **Theorem**: Show that the Feasibility Of Quadratic Equations (FQE) is NP-complete
- **Proof**: We consider the Reduction:
  - STABLE SET → FQE
  - STABLE SET is NP-complete
  - Hence FQE is NP-complete
  - **But how can I make the Reduction?**
  - I only know STABLE SET → IP

STABLE SET



$\alpha(G) \geq k$

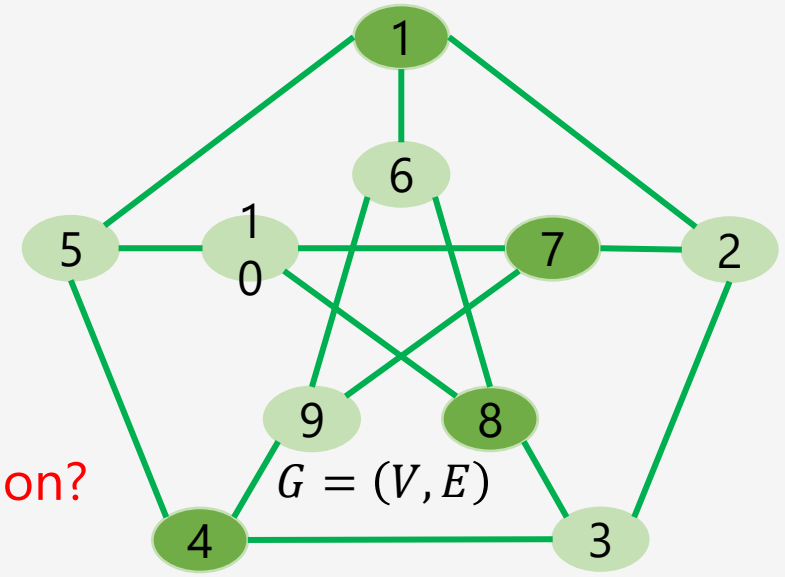
IP

- **Decision variables:**
  - $x_i = 1$, if node i is picked;
  - $x_i = 0$, otherwise
- **Objective:**
  - $\cancel{\max f(x) = \sum_{i=1}^{10} x_i}$
- **Constraints:**
  - $\sum_{i=1}^{10} x_i \geq k$
  - $x_i + x_j \leq 1, \forall (i,j) \in E$
  - $x_i \in \{0,1\}, \forall i \in V$

Feasibility of Quadratic Equations is NP-complete

- **Theorem**: Show that the Feasibility Of Quadratic Equations (FQE) is NP-complete
- **Proof**: We consider the Reduction:
  - STABLE SET → FQE
  - STABLE SET is NP-complete
  - Hence FQE is NP-complete
  - But how can I make the Reduction?
  - I only know STABLE SET → IP

STABLE SET



$\alpha(G) \geq k$

IP

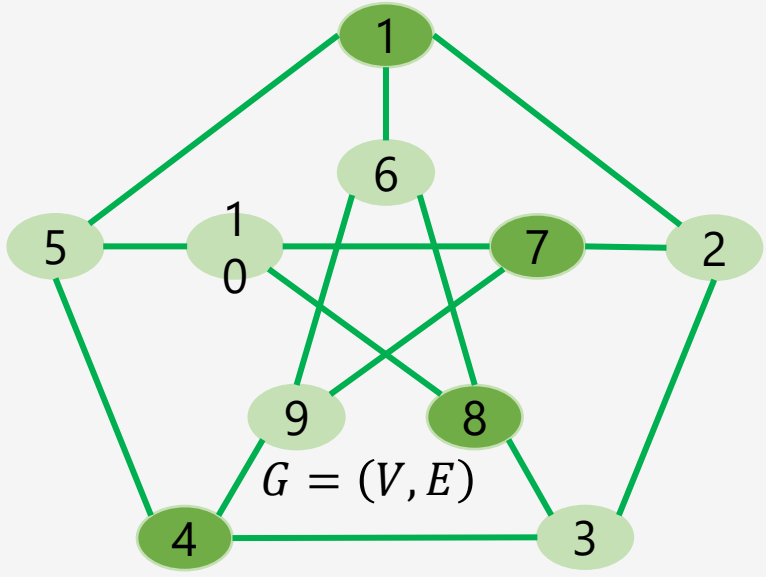
- **Decision variables:**
  - $x_i = 1$, if node i is picked;
  - $x_i = 0$, otherwise
- **Objective:**
  - $\cancel{\max f(x) = \sum_{i=1}^{10} x_i}$
- **Constraints:**
  - $\sum_{i=1}^{10} x_i \geq k$
  - $x_i + x_j \leq 1, \forall (i,j) \in E$
  - $x_i \in \{0,1\}, \forall i \in V$

Can you think how to convert the IP to a Quadratic IP ??

Feasibility of Quadratic Equations is NP-complete

- **Proof (cont.)**: I can write the IP as a FQE Decision problem and have IP →FQE

STABLE SET



$\alpha(G) \geq k$

→ IP

- **Decision variables:**
  - $x_i = 1$, if node i is picked;
  - $x_i = 0$, otherwise
- **Objective:**
  - $\max f(x) = \sum_{i=1}^{10} x_i$
- **Constraints:**
  - $\sum_{i=1}^{10} x_i \geq k$
  - $x_i + x_j \leq 1, \forall (i,j) \in E$
  - $x_i \in \{0,1\}, \forall i \in V$

→ FQE

- **Decision variables:**
  - $x_i \in [0,1]$,
  - $s \in R$
- **Objective:**
  - $\max f(x) = \sum_{i=1}^{10} x_i$
- **Constraints:**
  - $\sum_{i=1}^{10} x_i - k = s^2$
  - $x_i x_j = 0, \forall (i,j) \in E$
  - $x_i(1 - x_i) = 0, \forall i \in V$

The POLYPOS Decision Problem

- <u>Input</u>: A multivariate polynomial $p: R^n \to R$, of degree m
- <u>Question</u>: Is there an $x \in R^n$ for which $p(x) > 0$??
  - Example: $p(x) = x_1^4 + 2x_1^3x_2 + 4x_1x_2^2 - x_1x_2x_3^2 + 5x_2^4 - x_3^3$
- We want to show that POLYPOS is NP-hard

The POLYPOS Decision Problem

- Input: A multivariate polynomial $p: R^n \to R$, of degree m
- Question: Is there an $x \in R^n$ for which $p(x) > 0$??
  - Example: $p(x) = x_1^4 + 2x_1^3x_2 + 4x_1x_2^2 - x_1x_2x_3^2 + 5x_2^4 - x_3^3$
- We want to show that POLYPOS is NP-hard
- We first need to ask if POLYPOS $\in NP$??

The POLYPOS Decision Problem

- Input: A multivariate polynomial $p: R^n \to R$, of degree m
- Question: Is there an $x \in R^n$ for which $p(x) > 0$??
  - Example: $p(x) = x_1^4 + 2x_1^3x_2 + 4x_1x_2^2 - x_1x_2x_3^2 + 5x_2^4 - x_3^3$
- We want to show that POLYPOS is NP-hard
- We first need to ask if POLYPOS $\in NP$??
  - Is there a simple certificate of the YES answer to the Question??
  - That is, if I give you $x^* \in R^n$ can you verify in Poly-time that $p(x^*) \leq 0$??

The POLYPOS Decision Problem

- Input: A multivariate polynomial $p: R^n \to R$, of degree m
- Question: Is there an $x \in R^n$ for which $p(x) > 0$??
  - Example: $p(x) = x_1^4 + 2x_1^3x_2 + 4x_1x_2^2 - x_1x_2x_3^2 + 5x_2^4 - x_3^3$
- We want to show that POLYPOS is NP-hard
- We first need to ask if POLYPOS $\in NP$??
  - Is there a simple certificate of the YES answer to the Question??
  - That is, if I give you $x^* \in R^n$ can you verify in Poly-time that $p(x^*) \leq 0$??
  - **We do not know! POLYPOS $\in NP$ is an open problem**

The POLYPOS Decision Problem

- Input: A multivariate polynomial $p: R^n \to R$, of degree m
- Question: Is there an $x \in R^n$ for which $p(x) \leq 0$??
  - Example: $p(x) = x_1^4 + 2x_1^3x_2 + 4x_1x_2^2 - x_1x_2x_3^2 + 5x_2^4 - x_3^3$
- We want to show that POLYPOS is NP-hard
- We first need to ask if POLYPOS $\in NP$??
  - Is there a simple certificate of the YES answer to the Question??
  - That is, if I give you $x^* \in R^n$ can you verify in Poly-time that $p(x^*) \leq 0$??
  - **We do not know! POLYPOS $\in NP$ is an open problem**

Consider the polynomial:

$$p(x) = (x_1 - 2)^2 + (x_2 - x_1^2)^2 + (x_3 - x_2^2)^2 + \cdots + (x_n - x_{n-1}^2)^2$$

The POLYPOS Decision Problem

- Input: A multivariate polynomial $p: R^n \to R$, of degree m
- Question: Is there an $x \in R^n$ for which $p(x) \leq 0$??
  - Example: $p(x) = x_1^4 + 2x_1^3x_2 + 4x_1x_2^2 - x_1x_2x_3^2 + 5x_2^4 - x_3^3$
- We want to show that POLYPOS is NP-hard
- We first need to ask if POLYPOS $\in NP$??
  - Is there a simple certificate of the YES answer to the Question??
  - That is, if I give you $x^* \in R^n$ can you verify in Poly-time that $p(x^*) \leq 0$??
  - **We do not know! POLYPOS $\in NP$ is an open problem**

Consider the polynomial:

$$p(x) = (x_1 - 2)^2 + (x_2 - x_1^2)^2 + (x_3 - x_2^2)^2 + \cdots + (x_n - x_{n-1}^2)^2$$

$$x^* = (2, 2^2, 2^4, \ldots, 2^{2^n})$$

The POLYPOS Decision Problem

- Input: A multivariate polynomial $p: R^n \to R$, of degree m
- Question: Is there an $x \in R^n$ for which $p(x) > 0$??
  - Example: $p(x) = x_1^4 + 2x_1^3x_2 + 4x_1x_2^2 - x_1x_2x_3^2 + 5x_2^4 - x_3^3$
- We want to show that POLYPOS is NP-hard
- We first need to ask if POLYPOS $\in NP$??
  - Is there a simple certificate of the YES answer to the Question??
  - That is, if I give you $x^* \in R^n$ can you verify in Poly-time that $p(x^*) \leq 0$??
  - **We do not know! POLYPOS $\in NP$ is an open problem**

Consider the polynomial:

$$p(x) = (x_1 - 2)^2 + (x_2 - x_1^2)^2 + (x_3 - x_2^2)^2 + \cdots + (x_n - x_{n-1}^2)^2$$

$$x^* = (2, 2^2, 2^4, \ldots, 2^{2^n})$$

We need 2^n number of bits to write the solution.

We cannot check the certificate in Poly-time

Hence POLYPOS $\notin NP$

PARTITION is NP-complete

- PARTITION:
  - Input: A list of positive integers $\{a_1, a_2, \dots, a_n\}$
  - Question: Can you split the list of integers into two bags such that the sum in one bag is the same as the sum in the other?

$S = \{5,2,1,6,3,8,5,4,1,1\}$

$S = \{5,2,1,6,3,8,5,4,1,1\}$

$5 + 3 + 8 + 1 + 1 = 18$

$2 + 1 + 6 + 5 + 4 = 18$

The YES is answer is easily verifiable $\Rightarrow$ PARTITION $\in NP$

But no one knowns how to efficiently verify a NO answer

PARTITION is NP-complete

- PARTITION:
  - Input: A list of positive integers $\{a_1, a_2, \dots, a_n\}$
  - Question: Can you split the list of integers into two bags such that the sum in one bag is the same as the sum in the other?

$S = \{5,2,1,6,3,8,5,4,1,1,1\}$

No partition exists for this set

$S = \{5,2,1,6,3,8,5,4,1,1,1\}$

$5 + 3 + 8 + 1 + 1 + 1 = 19$

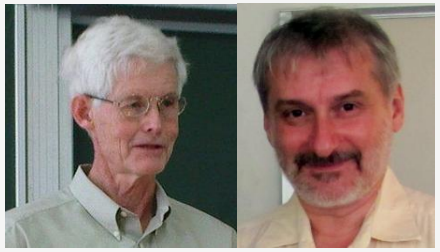
$2 + 1 + 6 + 5 + 4 = 18$

The YES is answer is easily verifiable $\Rightarrow$ PARTITION $\in NP$

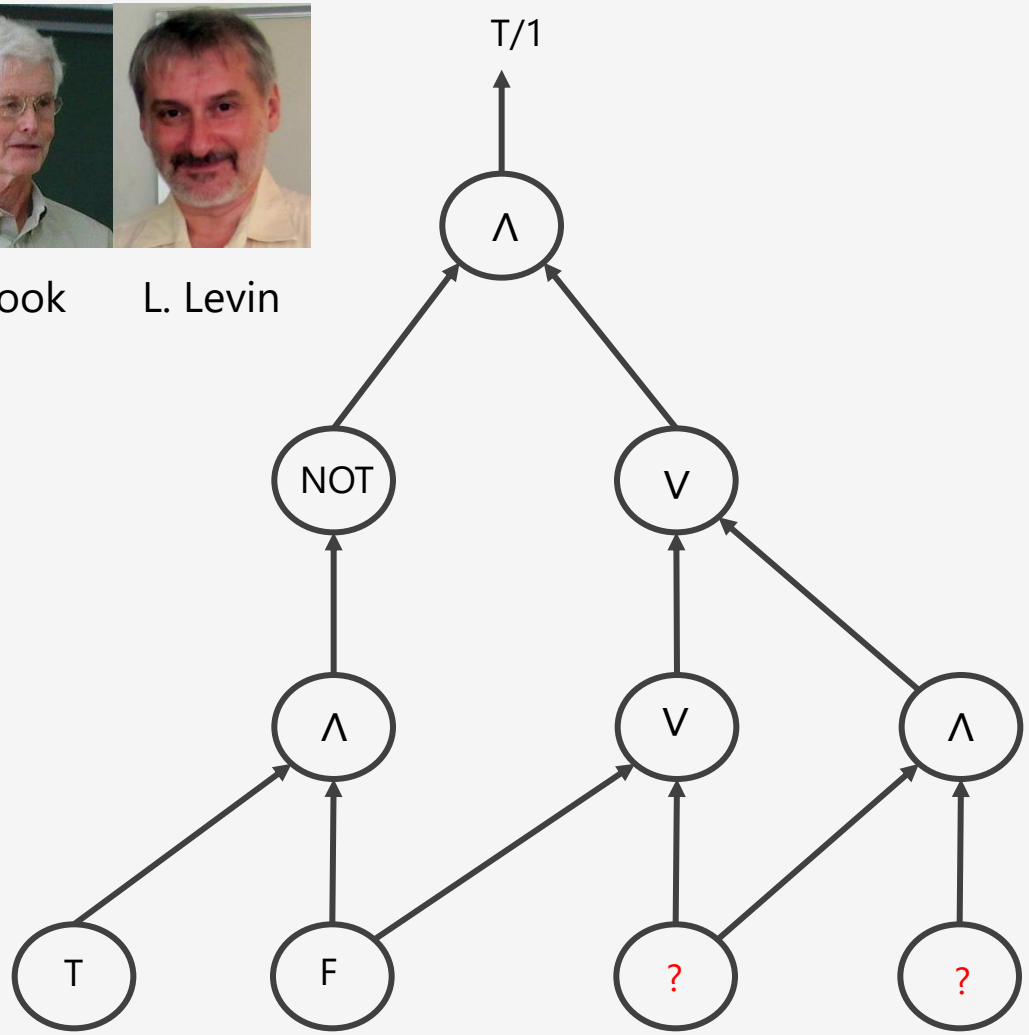
But no one knowns how to efficiently verify a NO answer

The first NP-complete problem

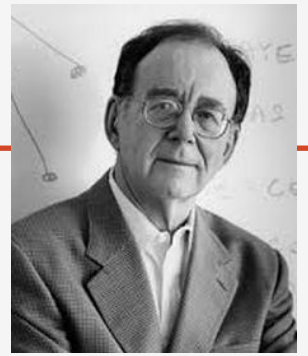
- CIRCUIT SAT
  - Input: A Boolean circuit C
  - Question: is there an assignment to the variables that cause the circuit to output to be True/1 ?
  - **Then the following Reductions were easy to show**: CIRCUIT SAT → SAT → 3SAT →…



S. Cook L. Levin



How did everything in $\mathbf{N}P - hard\ started$?



- Richard Karp showed that 21 Decision Problems are NP-hard.
- He first proved that SAT is NP-hard.
- Then he gave Reductions to the other 20

REDUCIBILITY AMONG COMBINATORIAL PROBLEMS†

Richard M. Karp

University of California at Berkeley

Abstract: A large class of computational problems involve the determination of properties of graphs, digraphs, integers, arrays of integers, finite families of finite sets, boolean formulas and elements of other countable domains. Through simple encodings from such domains into the set of words over a finite alphabet

First 21 NP-complete problems

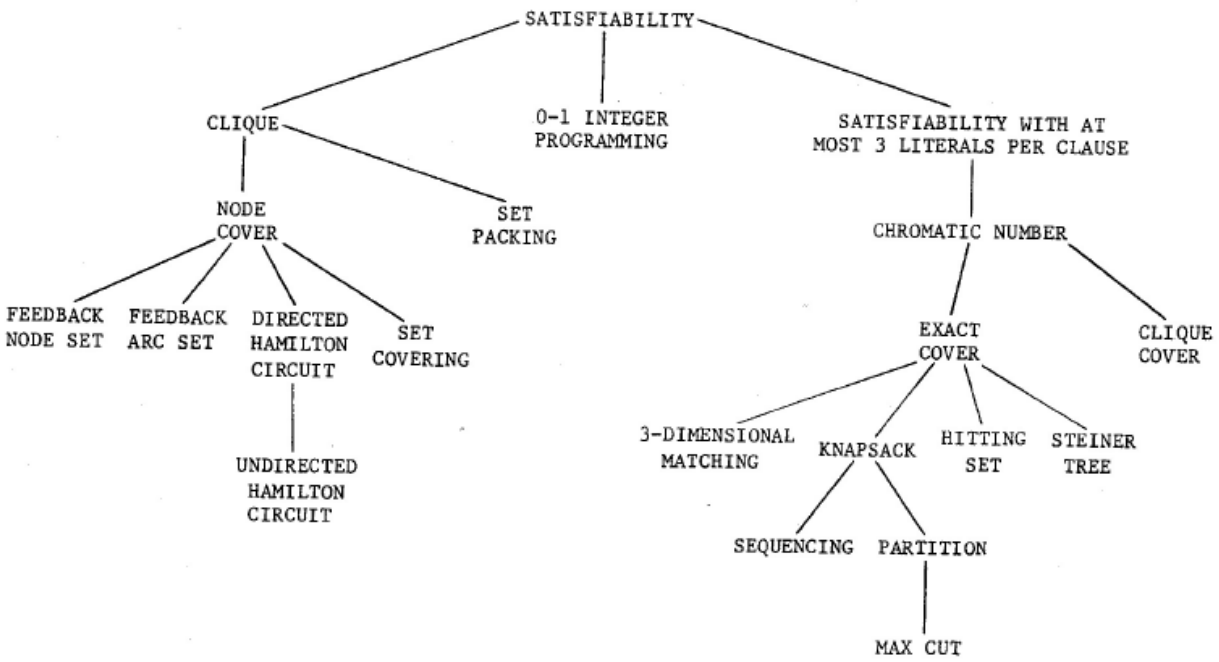
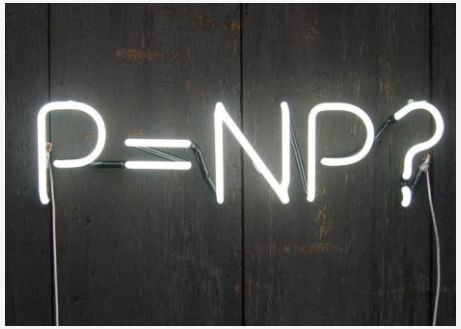


FIGURE 1 - Complete Problems

The Domino Effect!

- All NP-complete problems reduce to each other!
- If you can solve one in Poly-time, you solve ALL in Poly-time!
  - In that case you would have proved $P = NP$
  - You see your name in The New York Times
  - Your paper will be one of the most highly cited papers ever!
  - You become a millioner ($P = NP$ is worth $1M)



Check your proof carefully as there are many people who believe they have proved $P = NP$, but they have not!

Main message

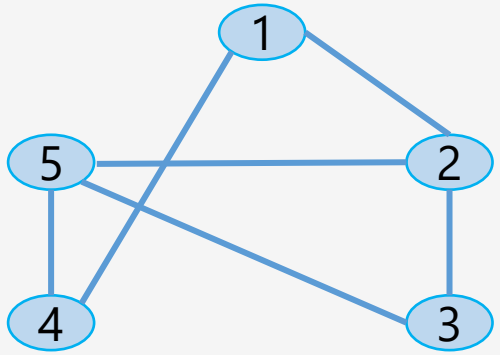
- Computational complexity theory beautifully classifies many problems of optimization theory as easy or hard
  - At the most basic level, easy means "in P", hard means "NP-hard"
    - MINCUT VS MAXCUT, PENONPAPER VS TSP, LP VS IP, …
- **Important**: when a problem is shown to be NP-hard, it does not mean that we should give up. NP-hard problems arise in applications all the time. There are good strategies for dealing with them.
  - Solving special cases exactly, or creating **Heuristics**
  - Using **Convex optimization** to compute bounds of near optimal solutions
- P=NP is still Open and $1M worth → **Maybe one day you guys will tell us you put a YES or NO on it**

Some help for your HW

The remaining slides provide hints for some of the problems in HW8.

Some help for HW8 / Prob. 1 (Social Networks application)

Consider the **smaller case** of Prof. CHEAPSKATE (only for 5 students)



Original graph - $G_1 = (V, E_1)$

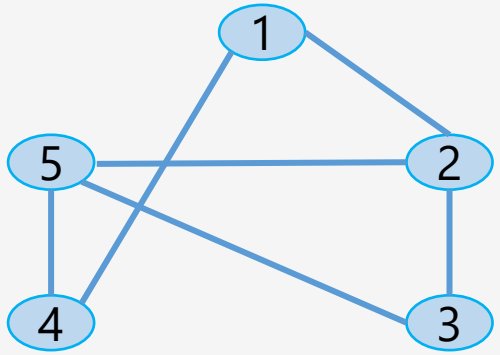
$$V = \{1,2,3,4,5\}$$

$$E_1 = \left\{ \begin{matrix} (1,4), (1,2), (2,5), (3,2), \\ (3,5), (4,5) \end{matrix} \right\}$$

Question: who are the students who are allowed to go to the party?

Some help for HW8 / Prob. 1 (Social Networks application)

Consider the smaller case of Prof. CHEAPSKATE (only 5 students)



Original graph - $G_1 = (V, E_1)$

$$V = \{1,2,3,4,5\}$$

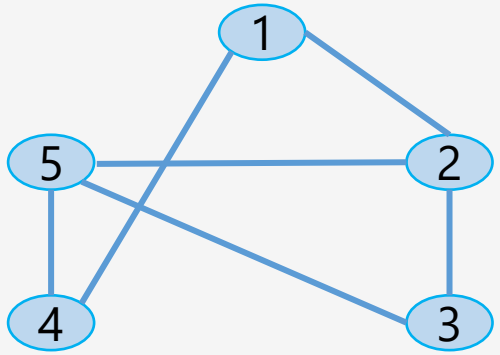
$$E_1 = \left\{ \begin{matrix} (1,4), (1,2), (2,5), (3,2), \\ (3,5), (4,5) \end{matrix} \right\}$$

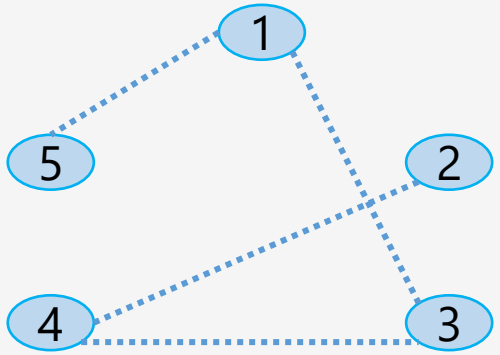
Question: who are the students who are allowed to go to the party? These people form a clique.

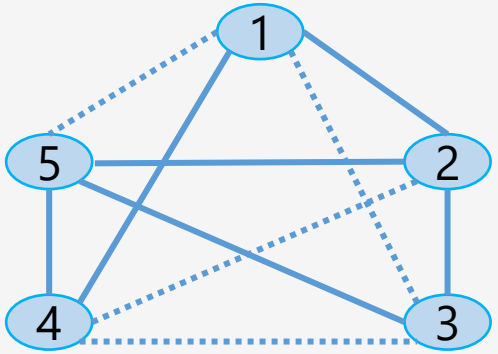
Do we know a way to model this problem as an Optimization problem??

Some help for HW8 / Prob. 1 (Social Networks application)

Consider the smaller case of Prof. CHEAPSKATE (only 5 students)







Original graph - $G_1 = (V, E_1)$

$$V = \{1,2,3,4,5\}$$

$$E_1 = \left\{ \begin{matrix} (1,4), (1,2), (2,5), (3,2), \\ (3,5), (4,5) \end{matrix} \right\}$$

Complement graph - $G_2 = (V, E_2)$

$$V = \{1,2,3,4,5\}$$

$$E_2 = \left\{ \begin{matrix} (1,5), (1,3), (2,4), (3,2), \\ (4,3) \end{matrix} \right\}$$

Complete graph - $G = (V, E)$

$$V = \{1,2,3,4,5\}$$

$$E = E_1 \cup E_2$$

Question: who are the students who are allowed to go to the party? These people form a clique.
Do we know a way to model this problem as an Optimization problem??

Find the maximum stable set of G_2

$$S = \{2,3,5\}$$

For HW8:
Prob1.a: Use LP and you will find 41.5
Prob1.b: Use SDP and you will find 11
(for the graph in Prob1).
They both provide Upper Bounds to $\alpha(G_2)$.
Only one is tight (equal) with $\alpha(G_2)$

Some help for HW8 / Prob. 1 (Social Networks application)

- **Prob1.c:** In the lectures we showed $X = \frac{1}{\alpha(G)} x x^T$, where x is the indicator vector of the maximum stable set (i.e., its nonzero elements are 1 and correspond to the vertices that belong to the max stable set)

Some help for HW8 / Prob. 1 (Social Networks application)

- **Prob1.c:** In the lectures we showed $X = \frac{1}{\alpha(G)} x x^T$, where x is the indicator vector of the maximum stable set (i.e., its nonzero elements are 1 and correspond to the vertices that belong to the max stable set)
- We also showed that X is feasible solution of the Lavasz SDP relaxation.
- We can show that X has rank=1.

$$
\begin{aligned}
SDP_{OPT} = \max \quad & Tr(JX) \\
\text{s.t.} \quad & Tr(X) = 1 \\
& X_{ij} = 0, \quad \forall (i,j) \in E \\
& X \succcurlyeq 0
\end{aligned}
$$

Some help for HW8 / Prob. 1 (Social Networks application)

- **Prob1.c:** In the lectures we showed $X = \frac{1}{\alpha(G)} x x^T$, where x is the indicator vector of the maximum stable set (i.e., its nonzero elements are 1 and correspond to the vertices that belong to the max stable set)

- We also showed that X is feasible solution of the Lavasz SDP relaxation.

$$
\begin{aligned}
SDP_{OPT} = \max \quad & Tr(JX) \\
\text{s.t.} \quad & Tr(X) = 1 \\
& X_{ij} = 0, \quad \forall (i,j) \in E \\
& X \succcurlyeq 0
\end{aligned}
$$

- We can show that X has rank=1.

  - To verify this, you can compute the eigenvalues of X and you will see that only one is non-zero. Let's say $\lambda \neq 0$.

Some help for HW8 / Prob. 1 (Social Networks application)

- **Prob1.c:** In the lectures we showed $X = \frac{1}{\alpha(G)} x x^T$, where x is the indicator vector of the maximum stable set (i.e., its nonzero elements are 1 and correspond to the vertices that belong to the max stable set)

- We also showed that X is feasible solution of the Lavasz SDP relaxation.

$$
\begin{aligned}
SDP_{OPT} = \max \quad & Tr(JX) \\
\text{s.t.} \quad & Tr(X) = 1 \\
& X_{ij} = 0, \quad \forall (i,j) \in E \\
& X \succcurlyeq 0
\end{aligned}
$$

- We can show that X has rank=1.

  - To verify this, you can compute the eigenvalues of X and you will see that only one is non-zero. Let's say $\lambda \neq 0$.

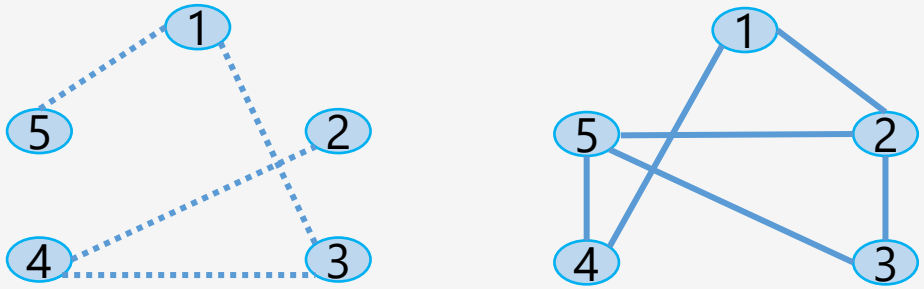
  - Now, consider the eigenvector v_λ that corresponds to λ,

  - What can the indices of the nonzero elements of v_λ tell you about the the students who are allowed to go to the professor's party??

Some help for HW8 / Prob. 1 (Social Networks application)

- **Prob1.d**: you need to show CHEAPSKATE is NP-complete
- Show first that CHEAPSKATE is NP-hard.
  - You need to show the reduction: STABLE SET → CHEAPSKATE
  - Follow the steps in (Prob1.a)
    - i. You are given a graph G_1 of all people knowing each other for ≥ 2 years
    - ii. You take its complement graph G_2
    - iii. Form the two decision problems with the same k

- **Prob1.d:** next show that CHEAPSKATE $\in NP$
  - You have to assume you are given a solution and check if there exists an edge between all pairs.
  - Is this easily verifiable process?.



STABLE SET (what do you know about its complexity?) → CHEAPSKATE

Reduction

Explain how you construct this reduction. Also make sure you explain if this reduction is efficient (i.e., it is polynomial in length)

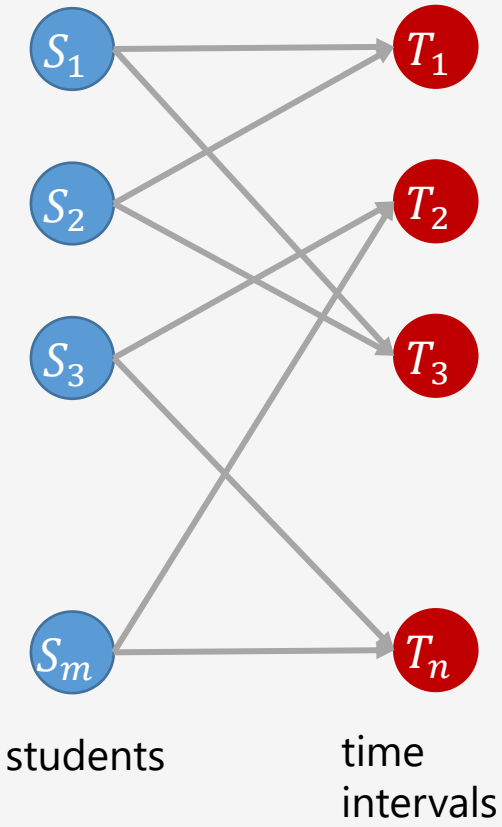
HW8 / Prob.2

- Professor Busy wants to schedule appointments for **19 students**
- She/He assigns **20 15-minutes slots**
- Students select the slots according to their preferences
- **Your Task**: Use LP to schedule non-overlapping meetings to each student
- Part (a): Prove the Reduction: SCHEDULING → MAXFLOW
- Part (b): DOODLE $\in P$
- Part (c): Solve an instance of the DOODLE problem as an LP using CVX

HW8 / Prob2

Part (a):

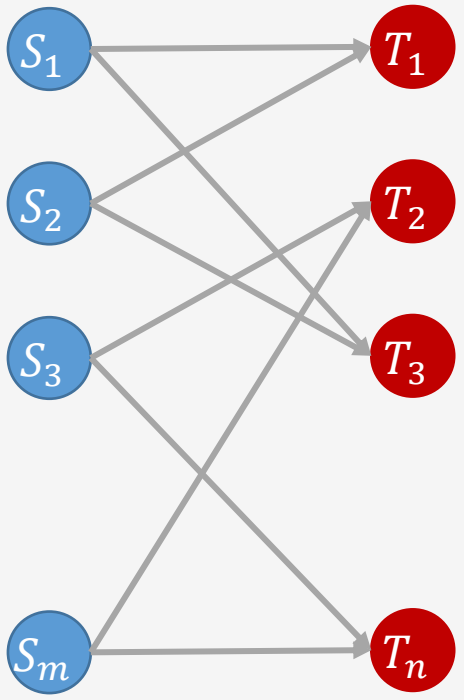
SCHEDULING



HW8 / Prob2

Part (a):

SCHEDULING

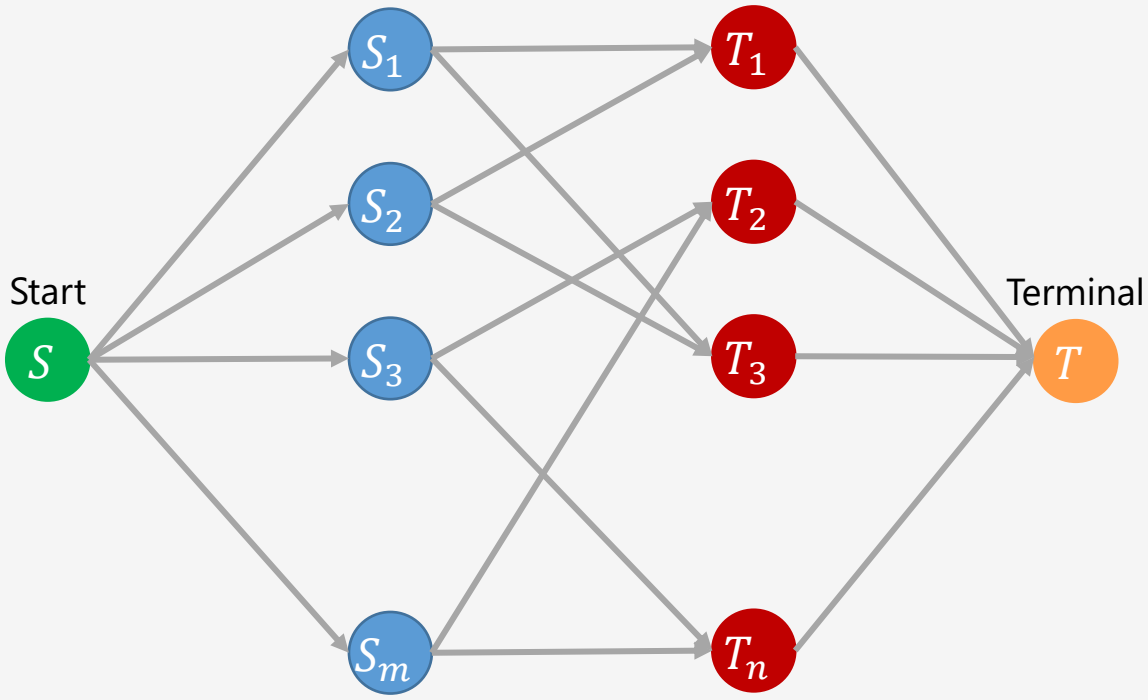


students

time intervals

- Some important questions (for the MAXFLOW):
  - What are the capacities??
  - What are the decision variables ??
  - What are your objective and constraints ??

MAXFLOW



students

time intervals

The Reduction is:

The scheduling of all the students is possible

$\Leftrightarrow$

There exists a max flow of size at least m

You have to show the equivalence

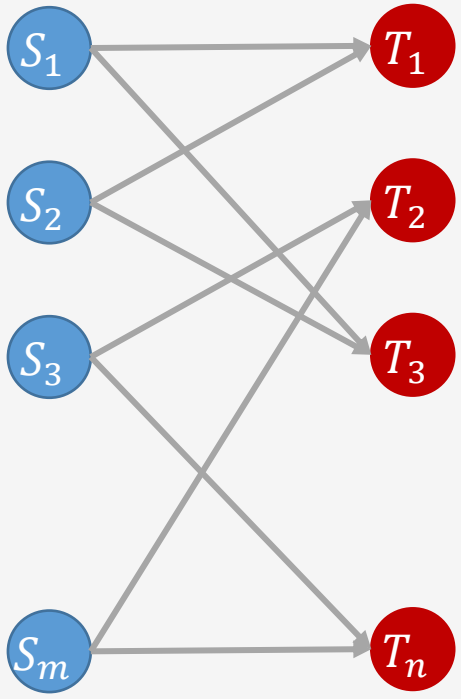
Sketch of the Proof:

($\Leftarrow$) use the capacities and the flow conservation rule: "what comes in a node" = "what comes out of a node" to show that the max flow is in fact a mapping of each student to their prefer time interval.

HW8 / Prob2

Part (a):

SCHEDULING

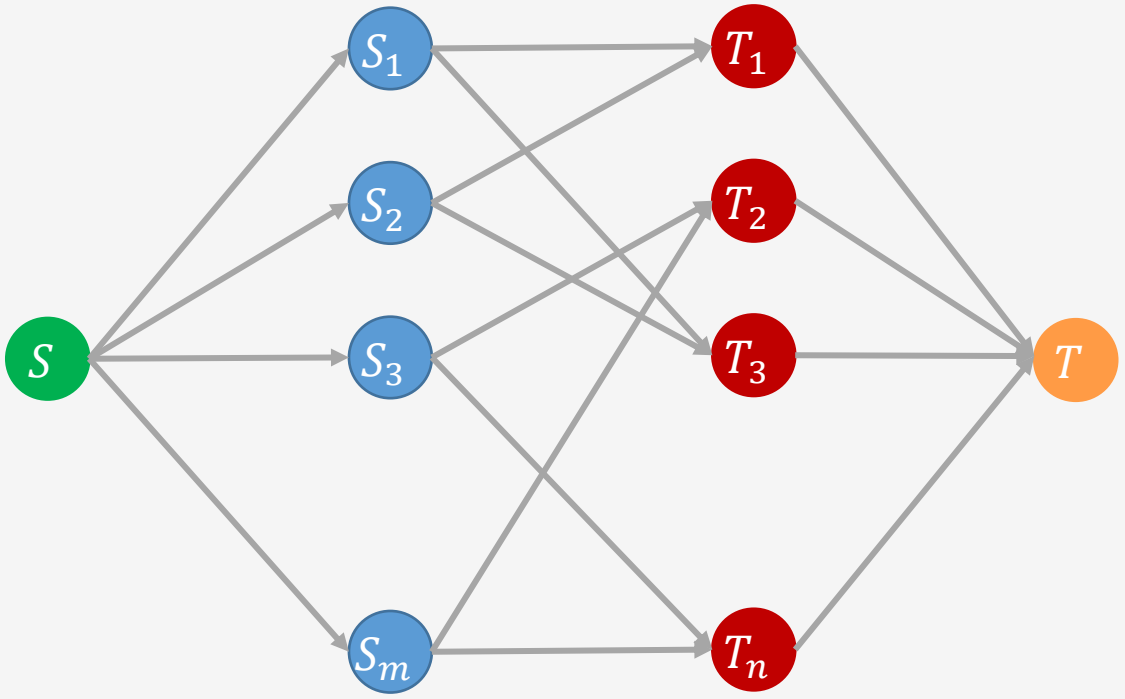


students

time intervals

- Some important questions (for the MAXFLOW):
  - What are the capacities??
  - What are the decision variables ??
  - What are your objective and constraints ??

MAXFLOW



students

time intervals

The reduction is:

The scheduling of all the students is possible

$\Leftrightarrow$

There exists a max flow of size at least m

You have to show the equivalence

Sketch of the Proof:
($\Rightarrow$) assume there is a schedule for all the students. Then you have to show that the max flow problem has a value equal to m.

How do you set the decision vars?

1. From S to the S_i ??
2. From S_i to T_j ??
3. From T_j to T ??

This will give you the optimal flow which matches the schedule

HW8 / Prob2

Part (c) asks you to formulate the MAXFLOW Problem as an **LP** and solve it for a specific instance (the adjacency matrix A for $m = 19$ students and $n = 20$ time intervals is given).

Once you solve the MAXFLOW Problem, you have to produce the schedule…

You can use the following decision variables:

1. x_i for $i = 1, \dots, m$ from the Source to the Student edges
2. y_{ij} for $i = 1, \dots, m$ and $j = 1, \dots, n$ for the Students to Times edges
3. z_j for $j = 1, \dots, n$ for the Times to Times to Terminal edges

HW8/Prob.3

- You are given the constrained optimization problem:

$$\begin{aligned} min \quad & p(x) = x_1^4 - 2x_1^3x_2 + x_1^2x_2^2 - 5x_1x_2^3 - x_2^4 \\ s.t. \quad & x_1^2 + x_2^2 = 0 \end{aligned}$$

- **Part (c)** asks you to generate 200 random points from the feasible set, and then evaluated the objective on all of them, and then select the one that gives you the smallest objective value
  - Make sure all the points you generate satisfy the constraint $x_1^2 + x_2^2 = 0$
  - One way to do this is the following procedure (repeat it 200 times):
    - Generate a random point $x = (x_1, x_2) \in R^2$
    - Compute $y_1 = \frac{x_1}{||x||}$ and $y_2 = \frac{x_2}{||x||}$. Then show that $y = (y_1, y_2)$ satisfies the constraint. Then evaluate the objective on y

HW8/Prob.3

- **Part (d)** asks you to show that if $\exists\gamma \in R$ such that
  - $p(x) - \gamma(x_1^2 + x_2^2)^2 = z(x)^T Q z(x)$ ..... (Eq.1)
  - for some $Q \succcurlyeq 0$ and $z(x) = (x_1^2, x_1 x_2, x_2^2)$
- Then you have to deduce that γ is a lower bound of $p(x)$.
- Note that:

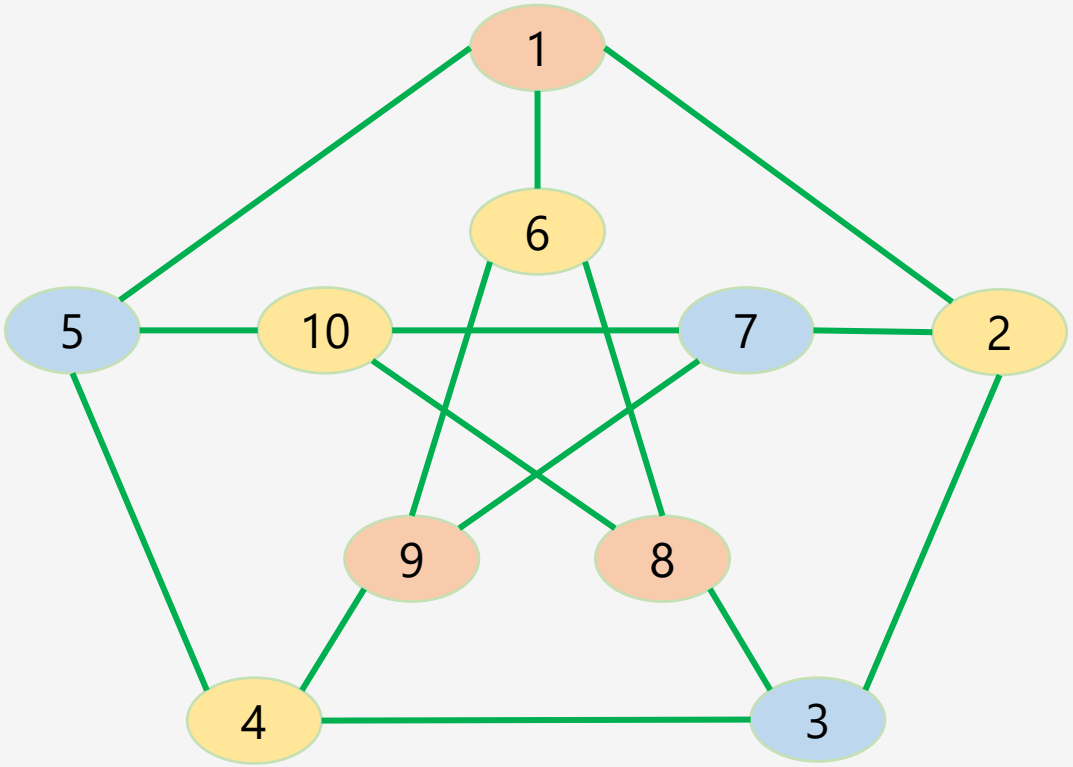
1. Since $Q \succcurlyeq 0$, we have $z(x)^T Q z(x) \geq 0$
2. Since $x_1^2 + x_2^2 = 1$ we have $(x_1^2 + x_2^2)^2 = 1$

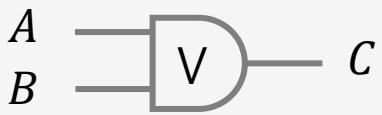
HW8/Prob.3

- **Part (e)** asks you to compute the largest γ, by solving an SDP using CVX.
  - Check Lecture nodes (Lec13.pdf) and Lecture slides Deck15 (they are both on Canvas (under Files/).
  - The SDP problem is a maximization problem (maximize γ),
  - You will also have several equality constraints so that (Eq.1) is satisfied
    - these equality constraints are derived by equating the coefficients of the left polynomial with the coefficients of the right polynomial in (Eq.1)
  - Your decision variables are: (i) γ, and (ii) the matrix Q which must be $\succcurlyeq 0$

Backup material

Backup material





$$\left[\quad\right]$$